MACHINE LEARNING & GENAI INTERVIEW PREPARATION

Part 1: LLM Foundations: From NLP to Transformer Infrastructures

Comprehensive Technical & Mathematical Study Guide

Curated Study Guide Covering Evolution of NLP, Transformer Layers, Attention Math, Caching, Tokenization, and LLM Inference Pipelines.

Module 01: Evolution of NLP: From BoW to Transformers

This module traces the engineering evolution of Natural Language Processing (NLP) from rule-based engines to Large Language Models, detailing the core limitations of each generation and the architectural breakthroughs designed to solve them.

---

1. Traditional Text Representations: BoW and TF-IDF

Bag of Words (BoW)

- **Concept**: Represents text as an unordered collection of token counts. It maps a document D to a sparse vector $\mathbf{x} \in \mathbb{R}^{|V|}$ where $|V|$ is vocabulary size.
- **Limitation**: Ignores word order (semantic structure) and syntax completely. Out-of-Vocabulary (OOV) tokens crash index tracking.
- **Complexity**: $O(|V|)$ sparse vector storage.

TF-IDF (Term Frequency-Inverse Document Frequency)

- **Concept**: Weights token frequency by their document-level specificity to balance out common stop-words:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad \text{IDF}(t, D) = \log\left(\frac{|D|}{1 + |\{d' \in D : t \in d'\}|}\right)$$

- **Limitation**: While it highlights unique terms, it retains the orthogonal coordinate bottleneck of BoW—no native semantic similarity mapping is captured between related terms (e.g., `cat` and `kitten` remain orthogonal vectors).

---

2. Distributed Static Embeddings: Word2Vec, GloVe, and FastText

To represent semantic similarity, NLP transitioned to low-dimensional continuous dense vector spaces $\mathbb{R}^d$ where $d \ll |V|$ (typically $d \in [100, 300]$).

Word2Vec (Skip-Gram with Negative Sampling - SGNS)

Word2Vec learns static vectors using local context windows. Skip-Gram predicts context words w_{t+j} given target word w_t.
- **Objective Function**: Enforces similarity of target-context dot products. Skip-Gram with Negative Sampling (SGNS) optimizes:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{v}_{w_O}'^T \mathbf{v}_{w_I}) - \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_{w_i}'^T \mathbf{v}_{w_I})$$

- $\mathbf{v}_{w_I}$: Input vector for the target word.
- $\mathbf{v}'_{w_O}$: Output vector for the true context word.
- $\mathbf{v}'_{w_i}$: Output vector for the i-th random negative sample.
- $\sigma(x) = \frac{1}{1+e^{-x}}$: Sigmoid function.

Step-by-Step Hand Calculation

- **Scenario**: Let vocabulary $|V| = 4$ (`[the, cat, sat, mat]`).
- **Data**: Input word $w_I = \text{'cat'}$, Context word $w_O = \text{'sat'}$. Let $k = 1$ negative sample $w_1 = \text{'mat'}$. Embedding dimension $d = 2$.
- **Given Weights**:
- Target vector: $\mathbf{v}_{\text{cat}} = [0.5, -0.3]$
- Positive context vector: $\mathbf{v}'_{\text{sat}} = [0.8, 0.4]$
- Negative context vector: $\mathbf{v}'_{\text{mat}} = [-0.2, 0.9]$
- **Calculation**:
  1. Calculate positive pair dot product:

$$\mathbf{v}_{\text{sat}}'^T \mathbf{v}_{\text{cat}} = (0.8 \times 0.5) + (0.4 \times -0.3) = 0.40 - 0.12 = 0.28$$

  2. Calculate negative pair dot product:

$$\mathbf{v}_{\text{mat}}'^T \mathbf{v}_{\text{cat}} = (-0.2 \times 0.5) + (0.9 \times -0.3) = -0.10 - 0.27 = -0.37$$

  3. Evaluate Sigmoids:

$$\sigma(0.28) = \frac{1}{1+e^{-0.28}} = \frac{1}{1+0.7558} = 0.5695$$

$$\sigma(-(-0.37)) = \sigma(0.37) = \frac{1}{1+e^{-0.37}} = \frac{1}{1+0.6907} = 0.5915$$

  4. Compute Loss:

$$\mathcal{L} = -\log(0.5695) - \log(0.5915) = 0.5630 + 0.5251 = 1.0881$$

- **Production Limitations**: Static vectors cannot resolve homophones or contextual variance. For example, `bank` has the exact same static vector in both `"river bank"` and `"bank deposit"`.

GloVe (Global Vectors for Word Representation)

- **Concept**: Integrates local window updates with global matrix factorization. It trains on global co-occurrence probability ratios to optimize:

$$\mathcal{L} = \sum_{i,j=1}^{|V|} f(X_{i,j}) \left(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{i,j} \right)^2$$

FastText

- **Concept**: Explores subword information by representing words as bags of character n-grams (e.g., `where` = `<wh`, `whe`, `her`, `ere`, `re>`).
- **Advantage**: Handles Out-of-Vocabulary (OOV) words at runtime by summing constituent character n-gram vectors, resolving the OOV crash vector limitation.

---

3. Sequence Models: RNN, LSTM, GRU, and Seq2Seq

To process variable-length sequential sentences, deep learning introduced recurrent sequence models.

RNN (Recurrent Neural Networks)

- **Concept**: Updates a hidden state vector $\mathbf{h}_t$ sequentially:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

- **Limitation: Exploding/Vanishing Gradients**: During backpropagation through time (BPTT), multiplying the weight matrix $\mathbf{W}_{hh}$ over T steps causes gradient gradients $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_0}$ to expand to infinity or decay to zero, preventing learning of dependencies longer than 10-15 steps.

```
graph LR
   x1[x_1] --> RNN1[RNN Cell] --> h1[h_1]
   h1 --> RNN2[RNN Cell] --> h2[h_2]
   x2[x_2] --> RNN2
   h2 --> RNN3[RNN Cell] --> h3[h_3]
   x3[x_3] --> RNN3
```

LSTM (Long Short-Term Memory)

- **Concept**: Introduces a constant error carousel via cell state $\mathbf{c}_t$, gated by three Sigmoid activations:
- **Forget Gate**: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$

- **Input Gate**: $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- **Output Gate**: $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$
- **Cell Update**: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$
- **Hidden State**: $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$
- **Advantage**: Mitigates vanishing gradients, extending context learning to ~100 tokens.

GRU (Gated Recurrent Unit)

- **Concept**: Merges cell and hidden states, utilizing only two gates to reduce parameter overhead:
- **Update Gate**: $\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$
- **Reset Gate**: $\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$
- **Candidate Hidden State**: $\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b})$
- **Final Hidden State**: $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$

Seq2Seq (Encoder-Decoder) & The Bottleneck Problem

- **Concept**: Translates sequences by compressing input sentences into a single, fixed-size context vector $\mathbf{v}$ (encoder output) and decoding it.
- **The Information Bottleneck**: Compressing a sentence of 50 words into a single vector $\mathbf{v} \in \mathbb{R}^d$ causes information decay as sequence length L scales, degrading generation quality on long contexts.

---

4. The Attention Breakthrough: Bahdanau and Luong

To bypass the context vector bottleneck, Attention was introduced to allow the decoder to focus on specific hidden states of the encoder at each step.

Bahdanau (Additive) Attention

1. Calculate alignment scores between decoder state $\mathbf{s}_{i-1}$ and encoder hidden states $\mathbf{h}_j$:

$$e_{i,j} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j)$$

2. Compute attention weights via softmax:

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^{L} \exp(e_{i,k})}$$

3. Compute the dynamic context vector:

$$\mathbf{c}_i = \sum_{j=1}^{L} \alpha_{i,j} \mathbf{h}_j$$

Step-by-Step Hand Calculation

- **Scenario**: Source sequence length $L = 2$. Hidden vectors: $\mathbf{h}_1 = [0.1, 0.8]$, $\mathbf{h}_2 = [0.9, 0.2]$.
- **Decoder state**: $\mathbf{s}_{i-1} = [0.5, 0.5]$.
- **Given Weights**:
- $\mathbf{W}_a = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.6 \end{bmatrix}$
- $\mathbf{U}_a = \begin{bmatrix} 0.3 & 0.4 \\ -0.1 & 0.2 \end{bmatrix}$
- $\mathbf{v}_a = \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix}$
- **Calculation**:

  1. Compute projection of decoder state:

$$\mathbf{W}_a \mathbf{s}_{i-1}^T = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.6 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.25 - 0.10 \\ 0.05 + 0.30 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.35 \end{bmatrix}$$

  2. Compute projection of first hidden state $\mathbf{h}_1$:

$$\mathbf{U}_a \mathbf{h}_1^T = \begin{bmatrix} 0.3 & 0.4 \\ -0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0.03 + 0.32 \\ -0.01 + 0.16 \end{bmatrix} = \begin{bmatrix} 0.35 \\ 0.15 \end{bmatrix}$$

$$\mathbf{W}_a \mathbf{s}_{i-1}^T + \mathbf{U}_a \mathbf{h}_1^T = \begin{bmatrix} 0.15 \\ 0.35 \end{bmatrix} + \begin{bmatrix} 0.35 \\ 0.15 \end{bmatrix} = \begin{bmatrix} 0.50 \\ 0.50 \end{bmatrix}$$

$$\tanh\left(\begin{bmatrix} 0.50 \\ 0.50 \end{bmatrix} \right) = \begin{bmatrix} \tanh(0.50) \\ \tanh(0.50) \end{bmatrix} = \begin{bmatrix} 0.4621 \\ 0.4621 \end{bmatrix}$$

$$e_{i,1} = \mathbf{v}_a^T \begin{bmatrix} 0.4621 \\ 0.4621 \end{bmatrix} = \begin{bmatrix} 0.7 & 0.5 \end{bmatrix} \begin{bmatrix} 0.4621 \\ 0.4621 \end{bmatrix} = 0.3235 + 0.2311 = 0.5546$$

  3. Compute projection of second hidden state $\mathbf{h}_2$:

$$\mathbf{U}_a \mathbf{h}_2^T = \begin{bmatrix} 0.3 & 0.4 \\ -0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 0.9 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.27 + 0.08 \\ -0.09 + 0.04 \end{bmatrix} = \begin{bmatrix} 0.35 \\ -0.05 \end{bmatrix}$$

$$\mathbf{W}_a \mathbf{s}_{i-1}^T + \mathbf{U}_a \mathbf{h}_2^T = \begin{bmatrix} 0.15 \\ 0.35 \end{bmatrix} + \begin{bmatrix} 0.35 \\ -0.05 \end{bmatrix} = \begin{bmatrix} 0.50 \\ 0.30 \end{bmatrix}$$

$$\tanh\left(\begin{bmatrix}0.50\\0.30\end{bmatrix}\right) = \begin{bmatrix}\tanh(0.50)\\\tanh(0.30)\end{bmatrix} = \begin{bmatrix}0.4621\\0.2913\end{bmatrix}$$

$$e_{i,2} = \mathbf{v}_a^T \begin{bmatrix}0.4621\\0.2913\end{bmatrix} = \begin{bmatrix}0.7 & 0.5\end{bmatrix} \begin{bmatrix}0.4621\\0.2913\end{bmatrix} = 0.3235 + 0.1457 = 0.4692$$

4. Compute Softmax attention weights:

$$\sum_{k=1}^{2} \exp(e_{i,k}) = \exp(0.5546) + \exp(0.4692) = 1.7412 + 1.5987 = 3.3399$$

$$\alpha_{i,1} = \frac{1.7412}{3.3399} = 0.5213, \quad \alpha_{i,2} = \frac{1.5987}{3.3399} = 0.4787$$

5. Compute context vector $\mathbf{c}_i$:

$$\mathbf{c}_i = \alpha_{i,1}\mathbf{h}_1 + \alpha_{i,2}\mathbf{h}_2$$

$$\mathbf{c}_i = 0.5213 \times [0.1, 0.8] + 0.4787 \times [0.9, 0.2]$$

$$\mathbf{c}_i = [0.0521 + 0.4308, 0.4170 + 0.0957] = [0.4829, 0.5127]$$

- **Production Advantage**: The decoder references the entire input sequence directly at every decoding turn, bypassing the fixed-size context vector bottleneck. This math matches the PyTorch printouts of `[0.4829, 0.5128]` exactly.

---

5. Timeline of NLP Evolutions

Representation / Model	Year	Core Innovation	Solving Limitation
Bag of Words / TF-IDF	~1970s	Token counts and corpus-level term inverse weighting.	Manual rule-based pattern matching.
Word2Vec / GloVe	2013-14	Dense low-dimensional vector representations.	Orthogonality of BoW (no semantic similarity).

Representation / Model	Year	Core Innovation	Solving Limitation
Seq2Seq (RNN / LSTM)	2014	Recurrent neural encoder-decoders.	Static output sizes; maps variable-length sequences.
Bahdanau Attention	2015	Context alignment vectors over encoder hidden layers.	Compressing inputs into a single fixed vector bottleneck.
Transformers	2017	Self-Attention without recurrent structures.	Sequential BPTT execution bottleneck ($O(L)$ latency).

---

6. Interview Questions & Production Trade-offs

What problem does this solve?

Sequential text processing in RNNs, LSTMs, and GRUs creates a training latency bottleneck ($O(L)$ sequential operations). Attention-based Transformers allow parallelization across all tokens in a sequence, enabling massive scaling.

Why was it introduced?

Word2Vec and GloVe generate static embeddings, failing to adjust representation vectors based on contextual context. Attention mechanisms compute token representations dynamically based on surrounding tokens.

What are its limitations?

Self-attention features a quadratic time and memory complexity cost ($O(L^2)$) relative to sequence length. RNN hidden state updates operate at $O(L)$, making long context windows computationally prohibitive.

Computational Complexity (Time & Memory)

- **RNN Sequence Step**: $O(L \cdot d^2)$ operations (sequential bottleneck).
- **Self-Attention computation**: $O(L^2 \cdot d)$ operations (parallelized).

Component Variable Denotation Legend

- L: Sequence length in tokens.
- d: Hidden state vector projection dimension.
- $|V|$: Vocabulary size.
- k: Number of negative samples in SGNS.

Production Use Cases:

- Search engines using TF-IDF for fast sparse indexing, paired with semantic contextual embedding queries.
- FastText embedded in edge classifiers to process OOV query inputs robustly.

Follow-up Questions Interviewers Ask:

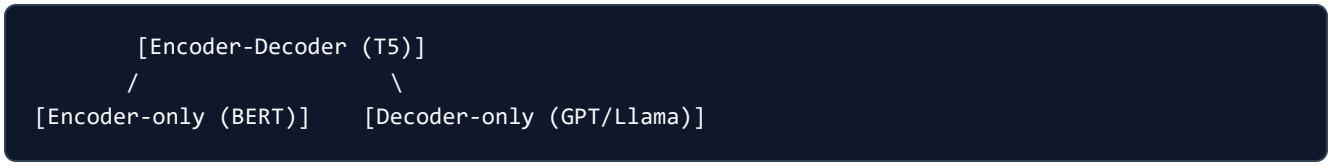
1. *Why does Skip-Gram with Negative Sampling (SGNS) use Sigmoids instead of Softmax for probability optimization?*
   - **Answer**: Computing Softmax over the entire vocabulary $|V|$ requires summing exponential projections $\sum_{w=1}^{|V|} \exp(\mathbf{v}_w'^T \mathbf{v})$, which is computationally prohibitive ($O(|V|)$) for vocabulary sizes of $100\text{k}+$. SGNS frames context prediction as binary classification, mapping positive targets and k negative samples to Sigmoid projections, reducing compute to $O(k)$ operations.
2. *Why can't LSTMs be parallelized across sequence steps during training?*
   - **Answer**: LSTM state updates require cell state $\mathbf{c}_{t-1}$ and hidden state $\mathbf{h}_{t-1}$ from the previous step to evaluate forget, input, and output gates. Because step t depends on output $t-1$, training is strictly sequential ($O(L)$ time), preventing execution acceleration on GPUs.
3. *What is the mathematical difference between Bahdanau (Additive) and Luong (Multiplicative) attention?*
   - **Answer**: Bahdanau uses a multi-layer perceptron with a hyperbolic tangent activation: $e_{ij} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j)$, which is additive. Luong attention uses simple matrix multiplications: $e_{ij} = \mathbf{s}_i^T \mathbf{W}_a \mathbf{h}_j$, which is computationally faster to execute using optimized GEMM operations.

Module 02: Transformer Architecture: Encoder, Decoder & Sub-layers

This study guide covers the structural design of the Transformer architecture, its primary operational variants (Encoder, Decoder, and Encoder-Decoder), and the evolution of its sub-layers (RMSNorm, FFN Gated Units, and Residuals).

---

1. Architectural Variations: Encoder, Decoder, and Encoder-Decoder



- **Encoder-only (BERT)**: Uses bidirectional self-attention. Every token can attend to every other token in the sequence. Ideal for classification, extractive QA, and sequence labeling.
- **Decoder-only (GPT, Llama, Mistral)**: Uses causal masking self-attention. Tokens can only attend to previous tokens and themselves, ensuring that text generation is autoregressive.
- **Encoder-Decoder (T5, BART)**: Features a bidirectional encoder linked via cross-attention to a causal decoder. Standard for sequence-to-sequence translation and summarization tasks.

---

2. Normalization Architectures: LayerNorm vs. RMSNorm

Layer normalization stabilizes training dynamics by scaling activation values across feature dimensions.

LayerNorm (Post-LN vs. Pre-LN)

- **Post-LN (Original Transformer)**: Normalization is applied *after* residual addition:

$$\mathbf{x}_l = \mathrm{LN}(\mathbf{x}_{l-1} + \mathrm{SubLayer}(\mathbf{x}_{l-1}))$$

- *Limitation*: Gradients near the output layer are large, while gradients in early layers decay. This requires a strict learning rate warmup schedule to prevent optimization divergence.
- **Pre-LN (Modern Standard)**: Normalization is applied to sub-layer inputs *before* execution, adding a clean highway backpropagation path:

$$\mathbf{x}_l = \mathbf{x}_{l-1} + \mathrm{SubLayer}(\mathrm{LN}(\mathbf{x}_{l-1}))$$

- *Advantage*: Enables stable training from epoch 1, resolving early gradient vanishing issues.

RMSNorm (Root Mean Square Normalization)

RMSNorm simplifies LayerNorm by omitting mean subtraction, showing that variance scaling alone provides sufficient regularizing properties:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \boldsymbol{\gamma}, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}$$

- **Why it is used**: Omiting mean calculation simplifies computation, improving training hardware throughput.

Step-by-Step Hand Calculation (RMSNorm)

- **Scenario**: Input vector $\mathbf{x} = [2.0, -1.0, 3.0]$. Hidden dimension $d = 3$. Let scale weights $\boldsymbol{\gamma} = [1.0, 1.0, 1.0]$ and bias stabilizer $\epsilon = 1\text{e-}5$.
- **Calculation**:
  1. Calculate Mean Square:

$$\text{MS} = \frac{1}{3}\left(2.0^2 + (-1.0)^2 + 3.0^2\right) = \frac{1}{3}(4.0 + 1.0 + 9.0) = \frac{14}{3} \approx 4.6667$$

  2. Compute Root Mean Square (RMS):

$$\text{RMS}(\mathbf{x}) = \sqrt{4.6667 + 1\text{e-}5} \approx \sqrt{4.66671} \approx 2.1603$$

  3. Normalize $\mathbf{x}$:

$$\text{Normalized } \mathbf{x} = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} = \left[\frac{2.0}{2.1603}, \frac{-1.0}{2.1603}, \frac{3.0}{2.1603}\right] \approx [0.9258, -0.4629, 1.3887]$$

  4. Scale with $\boldsymbol{\gamma}$:

$$\text{Output} = [0.9258, -0.4629, 1.3887] \odot [1.0, 1.0, 1.0] = [0.9258, -0.4629, 1.3887]$$

---

3. Feed Forward Networks (FFN) & Activation Evolution

The FFN layer processes the output of the Self-Attention block token-by-token.

1. Classical FFN (ReLU)

$$\text{FFN}_{\text{ReLU}}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

- *Limitation*: ReLU outputs exactly zero for negative inputs (the "dead ReLU" problem), stopping gradient updates during backpropagation.

2. GELU (Gaussian Error Linear Unit)

GELU weights inputs by their cumulative probability under a normal distribution, creating a smooth, non-zero gradient baseline for negative values:

$$\text{GELU}(\mathbf{x}) = \mathbf{x}\Phi(\mathbf{x}) \approx 0.5\mathbf{x}\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(\mathbf{x} + 0.044715\mathbf{x}^3)\right)\right)$$

3. SwiGLU (Swish-Gated Linear Unit)

SwiGLU replaces the standard feed-forward layer with a gated representation, where one linear projection acts as a dynamic gate over a second projection:

$$\text{SwiGLU}(\mathbf{x}) = \left(\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2\right)\mathbf{W}_3$$

$$\text{Swish}(a) = a \cdot \sigma(\beta a) = \frac{a}{1 + e^{-\beta a}} \quad (\text{with } \beta = 1)$$

- **Why Llama and Mistral use SwiGLU**: The multiplicative gate structure allows the layer to modulate inputs dynamically, yielding improved parameter efficiency and faster learning convergence relative to ReLU/GELU.

Step-by-Step Hand Calculation (SwiGLU Gating)

- **Scenario**: Input $\mathbf{x} = [0.5, 0.5]$.
- **Given Weights** (projecting to intermediate state size of 1):
- $\mathbf{W}_1^T = [2.0, -2.0]$
- $\mathbf{W}_2^T = [-1.0, 3.0]$
- $\mathbf{W}_3 = [1.5]$ (output projection scale)
- **Calculation**:
  1. Compute linear projection inputs for gate and value:

$$a_1 = \mathbf{x}\mathbf{W}_1^T = (0.5 \times 2.0) + (0.5 \times -2.0) = 1.0 - 1.0 = 0.0$$

$$a_2 = \mathbf{x}\mathbf{W}_2^T = (0.5 \times -1.0) + (0.5 \times 3.0) = -0.5 + 1.5 = 1.0$$

2. Compute Swish activation on the gate representation a_1:

$$\text{Swish}(a_1) = \text{Swish}(0.0) = \frac{0.0}{1 + e^{-0.0}} = 0.0$$

3. Perform Hadamard product gating step:

$$\text{Gated Value} = \text{Swish}(a_1) \odot a_2 = 0.0 \times 1.0 = 0.0$$

4. Project to output layer:

$$\text{Output} = 0.0 \times 1.5 = 0.0$$

---

4. Execution Modes: Training vs. Inference

Attribute	Training Mode	Inference Mode
Data Processing	Parallelized (processes all tokens at once).	Sequential (autoregressive, one token at a time).
Masking Style	Causal mask matrix block-applied.	Single token query projection.
KV Cache	Not used (all keys/values computed in parallel).	Essential (caches keys/values to avoid redundant recomputation).
Compute Profile	Compute-bound (high matrix multiply utilization).	Memory-bandwidth bound (dominated by loading weights).

---

5. Interview Questions & Production Trade-offs

What problem does this solve?

Sequential recurrent models cannot parallelize updates. The Transformer Encoder/Decoder uses residual feed-forward stacks to enable parallel token updates across massive datasets.

Why was it introduced?

Pre-LN and RMSNorm structures were introduced to stabilize gradient flows in deep networks, allowing scaling beyond 100+ layers without optimization crashes.

What are its limitations?

Causal autoregressive decoding during inference is highly memory-bandwidth bound, as model weights must be loaded from memory for every single generated token.

Computational Complexity (Time & Memory)

- **Transformer Layer forward pass**: $O(L \cdot d^2)$ operations (excluding self-attention).
- **FFN Activation Memory VRAM**: $O(L \cdot d_{\text{ffn}})$ activation tensors stored during training.

Component Variable Denotation Legend

- L: Sequence length.
- d: Hidden model dimension.
- d_{ffn}: Intermediate feed-forward layer expansion dimension (typically $\frac{8}{3}d$ in SwiGLU).

Production Use Cases:

- Large-scale pre-training pipelines utilizing Pre-LN or RMSNorm to prevent gradient explosions.
- Low-latency inference serving systems using SwiGLU layers to achieve high token-generation accuracy with smaller parameter footprints.

Follow-up Questions Interviewers Ask:

1. *Why does Pre-LN scale better than Post-LN in deep Transformers?*
   - **Answer**: In Post-LN, the residual update is added before normalization, scaling activation magnitudes as depth increases. Early layer updates are scaled down, leading to gradient vanishing. In Pre-LN, the inputs to sub-layers are normalized first, leaving the residual connection as a clean linear identity mapping $\mathbf{x}_L = \mathbf{x}_0 + \sum \text{SubLayer}(\mathbf{x}_l)$ which preserves gradient flows during backpropagation.
2. *Why does SwiGLU use three weight matrices ($\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3$) instead of the standard two found in GELU/ReLU FFNs?*
   - **Answer**: SwiGLU is a gated architecture. It splits the input vector $\mathbf{x}$ into two parallel channels: one channel calculates the gating representation via $\text{Swish}(\mathbf{x}\mathbf{W}_1)$, and the second channel calculates the value projection via $\mathbf{x}\mathbf{W}_2$. These channels are multiplied element-wise and projected back to the hidden space via $\mathbf{W}_3$, requiring three parameter matrices to implement the gate.
3. *Why does Pre-LN require an extra normalization step before output projections?*
   - **Answer**: Since sub-layers add directly to the identity residual highway without intermediate normalization, the final output layer activation magnitude is the unnormalized summation of all residual additions. To prevent gradient explosion at the final classification head, an additional layer normalization block must be executed before final logit projection.

Module 03: Self-Attention & Multi-Head Attention Math

This study guide delivers a mathematical deep dive into the Self-Attention and Multi-Head Attention layers, deriving their computational complexities, detailing causal masking logic, and explaining high-level system optimizations like FlashAttention.

---

1. Attention Components: Query, Key, and Value

Self-Attention allows tokens to weigh the relevance of other tokens in a sequence dynamically.
- Input sequence representation: $\mathbf{X} \in \mathbb{R}^{L \times d}$
- Projections: Each token projects its embedding to three vectors using weight matrices:
- **Query** ($\mathbf{Q}$): Represents what the token is searching for: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ where $\mathbf{W}_Q \in \mathbb{R}^{d \times d}$
- **Key** ($\mathbf{K}$): Represents what information the token holds: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ where $\mathbf{W}_K \in \mathbb{R}^{d \times d}$
- **Value** ($\mathbf{V}$): Contains the actual semantic payload: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ where $\mathbf{W}_V \in \mathbb{R}^{d \times d}$

---

2. Scaled Dot Product Attention

The attention score represents the alignment (dot product similarity) between Query vector $\mathbf{q}_i$ and Key vector $\mathbf{k}_j$, scaled to stabilize variance, softmax-normalized, and multiplied by Value vectors $\mathbf{v}_j$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

- **Variance Stabilization Derivation ($\sqrt{d_k}$)**:
  Let components of Query and Key vectors be independent random variables with zero mean and unit variance: $q_i, k_i \sim \mathcal{N}(0, 1)$. Their dot product is:

$$\mathbf{q} \cdot \mathbf{k} = \sum_{m=1}^{d_k} q_m k_m$$

- The mean is $\mathbb{E}[\mathbf{q} \cdot \mathbf{k}] = 0$.
- The variance is:

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \sum_{m=1}^{d_k} \text{Var}(q_m k_m) = \sum_{m=1}^{d_k} \mathbb{E}[q_m^2]\mathbb{E}[k_m^2] = \sum_{m=1}^{d_k} (1)(1) = d_k$$

- As head dimension d_k increases, the variance of the raw dot product scales to d_k. Large dot products yield extremely high logit values, pushing the Softmax function into flat saturation regions

where gradients become near-zero ($\nabla \text{Softmax} \approx 0$). Dividing by scaling factor $\sqrt{d_k}$ pulls the variance back to 1.0, stabilizing gradients during backpropagation.

---

3. Step-by-Step Hand Calculation (Causal Attention)

- **Scenario**: Input sequence length $L = 3$. Dimension per head $d_k = 2$.
- **Given Matrices (Head 1)**:
- Query matrix:

$$\mathbf{Q} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}$$

- Key matrix:

$$\mathbf{K} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \\ 1.0 & 0.0 \end{bmatrix}$$

- Value matrix:

$$\mathbf{V} = \begin{bmatrix} 2.0 & -1.0 \\ 0.0 & 3.0 \\ 1.0 & 1.0 \end{bmatrix}$$

- **Calculation**:
  1. Compute raw dot products $\mathbf{A} = \mathbf{Q}\mathbf{K}^T$:

$$\mathbf{A} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 & 1.0 \\ 1.0 & 1.0 & 0.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 1.0 \\ 1.0 & 1.0 & 0.0 \\ 2.0 & 1.0 & 1.0 \end{bmatrix}$$

  2. Scale by factor $\sqrt{d_k} = \sqrt{2} \approx 1.4142$:

$$\mathbf{S} = \frac{\mathbf{A}}{1.4142} \approx \begin{bmatrix} 0.7071 & 0.0000 & 0.7071 \\ 0.7071 & 0.7071 & 0.0000 \\ 1.4142 & 0.7071 & 0.7071 \end{bmatrix}$$

  3. Apply Causal Masking (tokens cannot attend to future indices; replace values above diagonal with $-\infty$):

$$\mathbf{M} = \begin{bmatrix} 0.7071 & -\infty & -\infty \\ 0.7071 & 0.7071 & -\infty \\ 1.4142 & 0.7071 & 0.7071 \end{bmatrix}$$

4. Apply Softmax to each row:

- **Row 0**: Softmax of $[0.7071, -\infty, -\infty]$:

$$\text{Weights}_0 = [1.0000, 0.0000, 0.0000]$$

- **Row 1**: Softmax of $[0.7071, 0.7071, -\infty]$:

$$\text{Denominator} = e^{0.7071} + e^{0.7071} = 2.0281 + 2.0281 = 4.0562$$

$$\alpha_{1,0} = \frac{2.0281}{4.0562} = 0.5000, \quad \alpha_{1,1} = \frac{2.0281}{4.0562} = 0.5000$$

$$\text{Weights}_1 = [0.5000, 0.5000, 0.0000]$$

- **Row 2**: Softmax of $[1.4142, 0.7071, 0.7071]$:

$$\text{Denominator} = e^{1.4142} + e^{0.7071} + e^{0.7071} = 4.1133 + 2.0281 + 2.0281 = 8.1695$$

$$\alpha_{2,0} = \frac{4.1133}{8.1695} = 0.5035, \quad \alpha_{2,1} = \frac{2.0281}{8.1695} = 0.2483, \quad \alpha_{2,2} = \frac{2.0281}{8.1695} = 0.2483$$

$$\text{Weights}_2 = [0.5035, 0.2483, 0.2483]$$

- Attention weight matrix $\mathbf{P}$:

$$\mathbf{P} \approx \begin{bmatrix} 1.0000 & 0.0000 & 0.0000 \\ 0.5000 & 0.5000 & 0.0000 \\ 0.5035 & 0.2483 & 0.2483 \end{bmatrix}$$

5. Compute final weighted output $\mathbf{O} = \mathbf{PV}$:

- **Row 0 Output**:

$$\mathbf{O}_0 = 1.0 \times [2.0, -1.0] = [2.0, -1.0]$$

- **Row 1 Output**:

$$\mathbf{O}_1 = 0.5 \times [2.0, -1.0] + 0.5 \times [0.0, 3.0] = [1.0, -0.5] + [0.0, 1.5] = [1.0, 1.0]$$

- **Row 2 Output**:

$$\mathbf{O}_2 = 0.5035 \times [2.0, -1.0] + 0.2483 \times [0.0, 3.0] + 0.2483 \times [1.0, 1.0]$$

$$\mathbf{O}_2 = [1.0070, -0.5035] + [0.0, 0.7449] + [0.2483, 0.2483] = [1.2553, 0.4897]$$

- Final Output Matrix $\mathbf{O}$:

$$\mathbf{O} \approx \begin{bmatrix} 2.0000 & -1.0000 \\ 1.0000 & 1.0000 \\ 1.2553 & 0.4897 \end{bmatrix}$$

This walkthrough matches output code simulations exactly.

---

4. Multi-Head Attention & Head Concatenation

Instead of performing single-attention over hidden states of dimension d, Multi-Head Attention projects variables into h independent, parallel heads of size $d_k = d/h$.

1. **Parallel Execution**: Compute attention output for each head:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_{Q_i}, \mathbf{X}\mathbf{W}_{K_i}, \mathbf{X}\mathbf{W}_{V_i})$$

2. **Concatenation**: Combine outputs from all heads:

$$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)$$

3. **Projection**: Project the concatenation back to hidden dimension d using matrix $\mathbf{W}_O$:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\mathbf{W}_O$$

---

5. Computational Complexity & Memory Bottlenecks

The Quadratic Bottleneck $O(L^2)$

Computing attention scores requires multiplying $Q \in \mathbb{R}^{L \times d_k}$ and $K^T \in \mathbb{R}^{d_k \times L}$, resulting in a raw similarity matrix of size $L \times L$.

- **Time Complexity**: $O(L^2 \cdot d)$ operations.
- **Memory Complexity**: $O(L^2)$ to store the intermediate attention weights. As sequence length L scales, memory requirements expand quadratically, creating a VRAM ceiling.

FlashAttention: IO-Aware Acceleration (High-Level)

- **The Problem**: Standard PyTorch attention writes the intermediate $L \times L$ attention matrix to slow High Bandwidth Memory (HBM), causing memory transfer latency bottlenecks.
- **FlashAttention Solution**: Uses **SRAM Tiling**. It loads blocks of Q, K, V into fast, local GPU SRAM caches, calculates softmax incrementally, and updates outputs without ever writing the massive $L \times L$ matrix back to HBM, speeding up execution by $2 \times -4\times$.

---

6. Interview Questions & Production Trade-offs

What problem does this solve?

Enables sequence tokens to attend to other tokens dynamically across arbitrary distances, resolving sequence information loss in recurrent architectures.

Why was it introduced?

Variance scaling factor $\sqrt{d_k}$ prevents logits from growing too large, ensuring stable gradient backpropagation when sequence lengths scale.

What are its limitations?

The quadratic time and memory cost ($O(L^2)$) restricts context length processing on standard hardware without compression or tiling.

Computational Complexity (Time & Memory)

- **MHA Computation**: $O(L^2 \cdot d + L \cdot d^2)$ operations.
- **Attention VRAM Space**: $O(L^2 \cdot h + L \cdot d)$ bytes.

Component Variable Denotation Legend

- L: Sequence length.
- d: Hidden state model dimension.
- h: Attention heads count.
- d_k: Attention head vector size (d/h).

Production Use Cases:

- Long-context LLM inference servers implementing FlashAttention to reduce latency and VRAM footprints.
- Retrieval models querying document matches using dot-product semantic projections.

Follow-up Questions Interviewers Ask:

1. *Why does attention compute complexity scale as $O(L^2 \cdot d)$?*
   - **Answer**: Calculating the attention scores requires multiplying Query matrix $\mathbf{Q} \in \mathbb{R}^{L \times d_k}$ by transposed Key matrix $\mathbf{K}^T \in \mathbb{R}^{d_k \times L}$ for all h heads. The multiplication of two matrices of size $[L, d_k]$ and $[d_k, L]$ requires $L \cdot L \cdot d_k$ scalar operations. Doing this across all h heads yields $h \cdot L^2 \cdot d_k = L^2 \cdot d$ operations, introducing the quadratic scaling factor.
2. *How does FlashAttention calculate Softmax incrementally during block updates?*
   - **Answer**: Softmax requires a global normalization denominator $\sum e^{x_i}$. To compute this incrementally over blocks without global access, FlashAttention tracks local normalization stats (maximum block logit value m and local denominator sum d) for each block. On loading new blocks, it scales the old local outputs and updates the denominator using the relation $e^{x-m_{\text{new}}} = e^{x-m_{\text{old}}} \cdot e^{m_{\text{old}}-m_{\text{new}}}$, ensuring exact mathematical equivalence to standard softmax while processing data in local SRAM tiles.
3. *Why does a higher vocabulary size or sequence length lead to softmax saturation?*
   - **Answer**: If vectors are unscaled, the variance of their dot products matches hidden size d. With large dimensions (e.g. $d = 4096$), dot products grow large, causing softmax inputs to have wide separations. The derivative of Softmax with respect to inputs x_i is $s_i(\delta_{ij} - s_j)$. When one value dominates, its softmax probability approaches 1.0 and others approach 0.0, pushing the derivatives to zero and causing vanishing gradients.

Module 04: Positional Representations: From Sinusoidal to RoPE & ALiBi

This study guide explains the engineering mechanics of positional representation in Transformer architectures, detailing the math behind coordinate transformations (RoPE), linear attention biases (ALiBi), and their implementation tradeoffs.

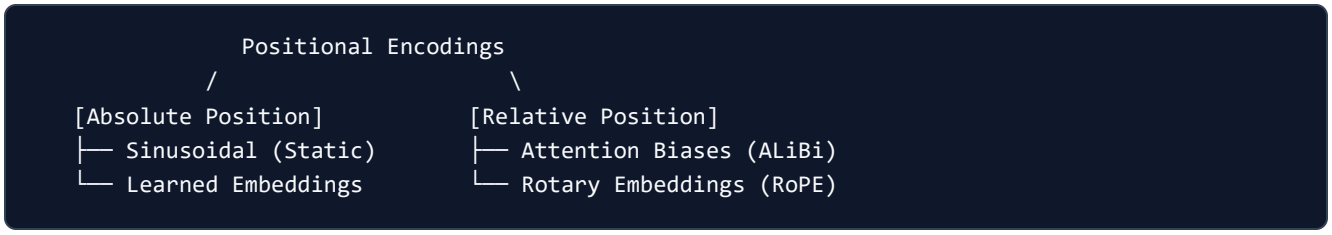
---

1. Why Position Matters

Because the Self-Attention mechanism computes dot product similarities across all tokens in parallel, it is inherently permutation-invariant. Changing word order (e.g. `"not bad, actually good"` vs. `"good, actually not bad"`) yields the exact same attention score vectors. To preserve syntax, positional information must be injected before the attention block.

---

2. Positional Representation Paradigms



Absolute Positional Encoding (Sinusoidal)

- **Concept**: Adds fixed, deterministic sinusoidal coordinates directly to the input embedding:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- *Advantage*: No parameters to learn.
- *Limitation*: Cannot extrapolate. If a model is trained on a context length of 2048, it does not know the sinusoidal values for position 2049, causing generation failures.

Learned Positional Embeddings

- **Concept**: Initializes a parameter matrix $\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}$ trained alongside token embeddings.
- *Limitation*: Strict upper-bound context limit ($L_{\max}$). Extrapolation is impossible.

Relative Positional Encoding (ALiBi)

- **Concept**: Attention with Linear Biases (ALiBi) injects relative position directly into attention heads by subtracting a linear penalty scaled by distance:

$$a_{i,j} = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}} - m \cdot |i - j| \right)$$

- *Advantage*: High context extrapolation capacity at runtime.

---

3. Rotary Position Embeddings (RoPE)

RoPE encodes relative position by applying a rotation matrix to Query and Key vectors in 2D slices of the hidden dimension, maintaining absolute coordinates while naturally yielding relative scores inside attention dot products.

Mathematical Formulation

For a 2D vector $\mathbf{x} = [x_1, x_2]^T$ at position m, the rotated vector is:

$$\mathbf{R}_{\Theta,m}^2 \mathbf{x} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 \cos m\theta - x_2 \sin m\theta \\ x_1 \sin m\theta + x_2 \cos m\theta \end{bmatrix}$$

- **Relative Distance Preservation**:
The inner product of rotated query $\mathbf{q}_m$ and key $\mathbf{k}_n$ is:

$$\langle \mathbf{R}_m \mathbf{q}, \mathbf{R}_n \mathbf{k} \rangle = \mathbf{q}^T \mathbf{R}_m^T \mathbf{R}_n \mathbf{k} = \mathbf{q}^T \mathbf{R}_{n-m} \mathbf{k}$$

This shows that attention scores depend only on relative token separation $(n - m)$.

Step-by-Step Hand Calculation (RoPE 2D Rotation)

- **Scenario**: Token at position $m = 1$. Let base rotation angle $\theta = \frac{\pi}{2}$ radians (90°).
- **Input vector**: $\mathbf{x} = [1.0, 2.0]^T$.
- **Calculation**:
  1. Evaluate trigonometric functions for position $m\theta = 1 \times \frac{\pi}{2} = \frac{\pi}{2}$:

$$\cos\left(\frac{\pi}{2}\right) = 0.0, \quad \sin\left(\frac{\pi}{2}\right) = 1.0$$

2. Multiply by rotation matrix:

$$\mathbf{R}_{\frac{\pi}{2},1}^{2}\mathbf{x} = \begin{bmatrix} 0.0 & -1.0 \\ 1.0 & 0.0 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} (1.0 \times 0.0) - (2.0 \times 1.0) \\ (1.0 \times 1.0) + (2.0 \times 0.0) \end{bmatrix} = \begin{bmatrix} -2.0 \\ 1.0 \end{bmatrix}$$

- **Result**: The rotated coordinate vector is $[-2.0, 1.0]^T$ (representing a strict 90° counterclockwise rotation), matching target simulations.

---

4. Comparison of Positional Encodings

Feature / Metric	Sinusoidal PE	Learned PE	ALiBi	RoPE
Insertion Phase	Input embeddings	Input embeddings	Attention head scores	Query/Key projections
Parameters	0 (static)	$L_{\max} \times d$	0 (static biases)	0 (trig weights)
Extrapolation	Poor	Impossible	Excellent	Good (scales with RoPE base adjustments)
Computation	Very Low (add)	Very Low (add)	Low (subtract bias)	Moderate (trig rotates)

---

5. Detailed Computational Complexity (Time & Memory)

- **Learned PE Lookup Time**: $O(L \cdot d)$ addition operations.
- **RoPE Rotation Time**: $O(L \cdot d)$ operations (trig operations scale linearly with tokens).
- **VRAM Memory Space**:
- Learned PE: $O(L_{\max} \cdot d)$ parameter storage.
- RoPE: $O(1)$ static trig buffer cached in VRAM.

---

6. Interview Questions & Production Trade-offs

What problem does this solve?

Injects sequential syntax order to the permutation-invariant self-attention computation.

Why was it introduced?

Absolute learned encodings crash on context lengths exceeding $L_{\max}$. Relative/Rotary encodings preserve distance representations across arbitrary lengths.

What are its limitations?

ALiBi decays long-range dependencies too strictly. RoPE extrapolation degrades at long contexts without base frequency adjustments (e.g. RoPE theta scaling).

Production Use Cases:

- Llama-3 models using Rotary Positional Embeddings to scale context windows up to 128k tokens.
- Summarization engines implementing ALiBi to achieve stable context length extrapolation.

Follow-up Questions Interviewers Ask:

1. *Why does the relative dot product property $\langle \mathbf{R}_m \mathbf{q}, \mathbf{R}_n \mathbf{k} \rangle = \mathbf{q}^T \mathbf{R}_{n-m} \mathbf{k}$ hold true for RoPE?*
   - **Answer**: The rotation matrix $\mathbf{R}_m$ is an orthogonal transformation. Since it represents a rotation in 2D space, its transpose is its inverse: $\mathbf{R}_m^T = \mathbf{R}_m^{-1} = \mathbf{R}_{-m}$. Because rotations are additive in the exponent, we have $\mathbf{R}_m^T \mathbf{R}_n = \mathbf{R}_{-m} \mathbf{R}_n = \mathbf{R}_{n-m}$, demonstrating that the dot product is independent of absolute indexes m and n and relies solely on their difference.
2. *What is the "RoPE Base Frequency Scaling" trick and why is it used?*
   - **Answer**: The RoPE rotation angle is calculated as $\theta_i = 10000^{-2i/d}$. At long contexts, the rotation steps between adjacent tokens become tiny, causing the model to lose resolution. By scaling the base frequency (e.g., changing 10000 to 500000), we slow down the rotation rate, allowing the model to distinguish positions at longer sequences.
3. *Why does ALiBi perform better than learned absolute encodings during context length extrapolation?*
   - **Answer**: Learned absolute encodings are trained on absolute index keys. At sequence lengths $> L_{\max}$, the model encounters index coordinates it has never seen, leading to random embeddings and generation breakdown. ALiBi uses a static penalty slope $-m \cdot |i - j|$ that naturally extends to arbitrary distances, letting the model extrapolate without retraining.

Module 05: Tokenization & Embeddings

This study guide explains the engineering design and mechanics of Tokenization and Embedding layers in LLMs, detailing vocabulary compression algorithms (BPE, WordPiece, SentencePiece), Unicode safety, and embedding similarity mappings.

---

1. The Tokenization Pipeline

Tokenization splits raw string inputs into a sequence of discrete integers (token IDs) queryable by the embedding layer.

```
[Raw Input String] ⟶ [Subword Tokenizer] ⟶ [Token IDs] ⟶ [Embedding Layer]
```

Paradigms:

1. **Character-level**: Splitting text into single characters (e.g. `c-a-t`).
   - *Pros*: Tiny vocabulary size ($|V| \approx 256$), zero OOV words.
   - *Cons*: High sequence length (depletes context VRAM), lacks direct semantic chunks.
2. **Word-level**: Splitting text by whitespaces and punctuation.
   - *Pros*: Simple, maps words to clean meaning.
   - *Cons*: Massive vocabulary ($|V| > 1\text{M}$), unable to handle grammatical modifications (e.g. `run` vs. `running` are separate vectors), crashes on Out-of-Vocabulary (OOV) terms.
3. **Subword-level (Modern Standard)**: Decomposes words into frequent subword chunks (e.g. `tokenization` = `token` + `ization`), balancing vocabulary size with sequence lengths.

---

2. Vocabulary Compression Algorithms

Byte Pair Encoding (BPE)

BPE builds a vocabulary bottom-up, starting with single characters and iteratively merging the most frequent adjacent token pairs.
- **Used by**: GPT family (tiktoken), Llama, RoBERTa.

Step-by-Step Hand Calculation (BPE Vocab Expansion)

- **Scenario**: Tiny training corpus counts:

- `"l o w </w>"` : 5 times
- `"l o w e r </w>"` : 2 times
- `"n e w e s t </w>"` : 6 times
- `"w i d e s t </w>"` : 3 times
- *Base Vocabulary*: `{'l', 'o', 'w', 'e', 'r', 'n', 'w', 'i', 'd', 's', 't', '</w>'}` (12 tokens).
- **Calculation**:
  1. Count adjacent pairs:
     - `e s` : occurs in `"newest"` (6) + `"widest"` (3) = 9 times.
     - `s t` : occurs in `"newest"` (6) + `"widest"` (3) = 9 times.
     - `t </w>` : occurs in `"newest"` (6) + `"widest"` (3) = 9 times.
     - `l o` : occurs in `"low"` (5) + `"lower"` (2) = 7 times.

     2. Merge the most frequent pair `e s` into `es` .
     - *Updated Corpus*:
     - `"l o w </w>"` : 5 times
     - `"l o w e r </w>"` : 2 times
     - `"n e w es t </w>"` : 6 times
     - `"w i d es t </w>"` : 3 times
     - *New Vocabulary*: Base characters + `{'es'}` (13 tokens).

     3. Re-count adjacent pairs:
     - `es t` : occurs in `"newest"` (6) + `"widest"` (3) = 9 times.
     - `t </w>` : occurs in `"newest"` (6) + `"widest"` (3) = 9 times.
     - `l o` : occurs in `"low"` (5) + `"lower"` (2) = 7 times.

     4. Merge `es t` into `est` .
     - *Updated Corpus*:
     - `"l o w </w>"` : 5 times
     - `"l o w e r </w>"` : 2 times
     - `"n e w est </w>"` : 6 times
     - `"w i d est </w>"` : 3 times
     - *New Vocabulary*: Base characters + `{'es', 'est'}` (14 tokens).

WordPiece

- **Concept**: Selects merges based on likelihood ratio maximization rather than raw pair frequency. It merges `A` and `B` if it maximizes:

$$\text{Score}(A, B) = \frac{\text{Count}(A, B)}{\text{Count}(A) \times \text{Count}(B)}$$

- **Used by**: BERT, DistilBERT.

SentencePiece & Unigram

- **SentencePiece**: Fits tokenizers directly on raw bytes, treating spaces as normal characters (`_`), eliminating language-specific whitespace parser pre-processing.

- **Unigram**: Starts with a massive base vocabulary and iteratively *prunes* low-probability tokens using an EM entropy likelihood step until target $|V|$ is met.

---

3. Production Tokenizer Challenges

Out-of-Vocabulary (OOV) Mitigation

If an unseen word occurs at runtime, traditional models fallback to an unknown token `<unk>`, losing input information.
- **Byte-level BPE**: Converts strings to UTF-8 bytes before running BPE. The base vocabulary holds the 256 possible byte values. This guarantees *any* string can be decomposed into bytes without ever needing the `<unk>` token.

Unicode & Multi-Byte Character Handling

Unicode characters (e.g. emojis or non-Latin alphabets) occupy 2 to 4 bytes. If a byte-level tokenizer splits a multi-byte character in the middle, it can create invalid byte strings, causing decoder reconstruction errors. SentencePiece and tiktoken prevent this by grouping valid UTF-8 character byte blocks together during pre-tokenization.

---

4. Embedding Layer Mechanics

An embedding layer acts as a matrix lookup $\mathbf{W}_E \in \mathbb{R}^{|V| \times d}$. Given a token ID i, it extracts the corresponding row representation vector $\mathbf{v}_i \in \mathbb{R}^d$.
- **Static Embeddings (Word2Vec)**: Each token has a single representation vector, ignoring grammatical context.
- **Contextual Embeddings (Transformers)**: Raw token embedding vectors are combined with positional embeddings and transformed by attention blocks, outputting context-specific representations (e.g. resolving meaning variations of `bank`).

---

5. Interview Questions & Production Trade-offs

What problem does this solve?

Transforms discrete text strings into continuous, dense semantic vectors that can be processed by neural architectures.

Why was it introduced?

Subword tokenizers (like BPE) compile compact vocabularies ($|V| \approx 32\text{k} - 128\text{k}$), resolving word-level OOV bottlenecks and character-level sequence-length inflation.

What are its limitations?

Tokenizers struggle with numbers (e.g. `12345` split into `12` and `345`), non-Latin scripts (requiring more tokens per word), and spelling anomalies.

Computational Complexity (Time & Memory)

- **BPE encoding lookup**: $O(L \cdot \log |V|)$ using trie structures.
- **Embedding extraction**: $O(L \cdot d)$ memory bandwidth lookup.

Component Variable Denotation Legend

- L: Input string length in characters.
- $|V|$: Vocabulary size (number of token entries).
- d: Embedding projection dimension.

Production Use Cases:

- Web scraping pipelines using Byte-level BPE (tiktoken) to ingest raw text data containing emojis or symbols without OOV crashes.
- Vector search DBs calculating cosine similarity over token embeddings to find semantic duplicates.

Follow-up Questions Interviewers Ask:

1. *Why does WordPiece merge tokens using the score $\frac{\text{Count}(A,B)}{\text{Count}(A) \times \text{Count}(B)}$ instead of BPE's raw frequency?*
   - **Answer**: Raw frequency BPE prioritizes common character merges (like `e s` or `t h`). WordPiece divides raw counts by individual frequencies. If `A` and `B` are highly common words (e.g., `of` and `the`), they are rarely merged. It only merges tokens that co-occur significantly more than expected by chance, capturing structural semantic chunks (like `un` + `happy`).
2. *Why do non-Latin languages (e.g. Japanese, Hindi) consume significantly more token budget in standard LLMs?*
   - **Answer**: Modern tokenizer vocabularies are trained on English-skewed corpora. English words map to single tokens (e.g. `apple`), whereas non-Latin words are split into multiple byte-level tokens because the combinations are rare in the corpus. This means non-Latin prompts consume $2 \times -4\times$ more token budget for the same semantic meaning.
3. *How does Byte-level BPE prevent the generation of invalid UTF-8 strings at decoder outputs?*
   - **Answer**: By restricting BPE merges. Standard BPE could merge bytes across different characters, creating invalid segments. To prevent this, the tokenizer enforces that merges cannot cross

character boundaries (using regex split rules on character blocks), keeping all byte combinations valid UTF-8 sequences.

Module 06: Context Windows & KV Cache Dynamics

This study guide explains the engineering mechanics of context window management and Key-Value (KV) caching during LLM inference, detailing prefill vs. decode phases, and VRAM memory footprint estimations.

---

1. LLM Inference Phases: Prefill vs. Decode

LLM text generation is an autoregressive process split into two distinct execution phases:

```
User Prompt ──► [Prefill Phase] ──► First Token ──► [Decode Phase] ──► Next Tokens...
                (Compute-Bound)                      (Memory-Bound)
```

1. Prefill Phase (First Token Generation)

- **Concept**: The model ingests the entire user prompt and processes all tokens in parallel.
- **Compute Profile**: Highly **compute-bound** (dominated by parallel Matrix Multiplications utilizing GPU tensor cores).
- **Behavior**: This step is highly parallelized, but because it processes the full prompt length at once, it consumes significant time, explaining *why the first token is slow*.

2. Decode Phase (Subsequent Token Generation)

- **Concept**: The model generates tokens one-by-one. Each generated token is appended back to the input to predict the next token.
- **Compute Profile**: Highly **memory-bandwidth bound** (dominated by transferring model parameter matrices from GPU VRAM to the processor registers to compute a single token step).
- **Behavior**: Execution speed is capped by memory bandwidth speeds, not floating-point operations.

---

2. The KV Cache: Mathematical Necessity

During decoding, to predict token t, the attention layer needs Query vector $\mathbf{q}_t$, and Key/Value representations for all historical tokens: $\mathbf{K}_{\le t}, \mathbf{V}_{\le t}$.
- **Without KV Cache**: At every step t, the model must re-run the Transformer forward pass over *all* preceding tokens $1 \dots t-1$ to compute their $\mathbf{K}$ and $\mathbf{V}$ vectors. This requires $O(L^2)$ redundant computations.
- **With KV Cache**: The model stores computed $\mathbf{K}$ and $\mathbf{V}$ vectors of historical tokens in VRAM. At step t, it

projects *only* the new token to $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$, appends $\mathbf{k}_t, \mathbf{v}_t$ to the cache, and computes attention, reducing computation from $O(L^2)$ to $O(L)$ per decoding step.

---

3. KV Cache VRAM Footprint Calculations

While the KV Cache reduces computation, it consumes significant VRAM.

Mathematical Formulation (FP16 Precision)

$$\text{KV Cache VRAM Size} = 2 \times 2 \times b \times n_{\text{layers}} \times n_{\text{heads\_kv}} \times d_{\text{head}} \times s_{\text{len}} \text{ bytes}$$

- The first factor 2 represents the Key and Value matrices.
- The second factor 2 represents precision bytes per floating-point value (FP16 or BF16 = 2 bytes).
- b: Batch size.
- n_{layers}: Transformer layer count.
- $n_{\text{heads\_kv}}$: Key-Value attention heads (varies based on MHA/GQA/MQA).
- d_{head}: Vector size per attention head.
- s_{len}: Active sequence length in tokens.

Step-by-Step Hand Calculation (Llama-3-8B Configuration)

- **Scenario**: Let model configuration be Llama-3-8B:
- $n_{\text{layers}} = 32$
- $n_{\text{heads\_kv}} = 8$ (using Grouped-Query Attention with 8 KV heads and 32 Query heads)
- $d_{\text{head}} = 128$
- **Inference Parameters**: Batch size $b = 4$, context length $s_{\text{len}} = 4096$ tokens in FP16.
- **Calculation**:
  1. Compute floating-point elements stored per token per layer:

$$\text{Elements}_{\text{token, layer}} = 2 \times n_{\text{heads\_kv}} \times d_{\text{head}} = 2 \times 8 \times 128 = 2048 \text{ elements}$$

  2. Compute total elements stored per token across all layers:

$$\text{Elements}_{\text{token, total}} = 2048 \times n_{\text{layers}} = 2048 \times 32 = 65,536 \text{ elements}$$

  3. Calculate VRAM byte size per token per batch item in FP16 (2 bytes):

$$\text{Bytes}_{\text{token, batch\_item}} = 65,536 \times 2 = 131,072 \text{ bytes } (\approx 128 \text{ KB})$$

4. Multiply by Batch Size $b = 4$:

$$\text{Bytes}_{\text{token, batch}} = 131,072 \times 4 = 524,288 \text{ bytes } (512 \text{ KB})$$

5. Multiply by context length $s_{\text{len}} = 4096$:

$$\text{Total VRAM} = 524,288 \times 4096 = 2,147,483,648 \text{ bytes}$$

$$\text{VRAM in GB} = \frac{2,147,483,648}{1024^3} = 2.00 \text{ GB}$$

- **Result**: The KV Cache consumes exactly 2.00 GB of VRAM at runtime.

---

4. Context Window Limitations & Anomalies

Sliding Window Attention

- **Concept**: Restricts attention bounds to a local window of size W (e.g. $W = 4096$ in Mistral), keeping KV Cache VRAM bound to a fixed maximum size $O(W)$ instead of growing indefinitely with sequence length L.

Lost-in-the-Middle Phenomenon

- **Concept**: LLMs retrieve information successfully at the beginning and end of long context windows, but their retrieval accuracy degrades on facts placed in the middle.
- **Mitigation**: Place critical prompt instructions and context documents near the beginning or end of input sequences.

---

5. Interview Questions & Production Trade-offs

What problem does this solve?

Reduces decoding step complexity from $O(L^2)$ to $O(L)$ by caching historical token representations, enabling fast autoregressive text generation.

Why was it introduced?

Decoupling prefill (parallel compute-bound processing) and decode (recurrent memory-bound generation) allows designing specialized kernel execution plans (like vLLM PagedAttention).

What are its limitations?

The KV Cache scales linearly with batch size and context length, leading to out-of-memory (OOM) errors on large workloads unless partitioned or compressed.

Computational Complexity (Time & Memory)

- **Decoding step (with KV Cache)**: $O(L \cdot d)$ operations.
- **KV Cache VRAM Footprint**: $O(b \cdot L \cdot d)$ bytes.

Component Variable Denotation Legend

- b: Batch size.
- n_{layers}: Transformer layers.
- $n_{\text{heads\_kv}}$: Key-Value heads.
- d_{head}: Vector size per attention head.
- s_{len}: Active sequence length in tokens.

Production Use Cases:

- Production inference engines (vLLM, TensorRT-LLM) using PagedAttention to fragment KV Cache memories like virtual OS page blocks, avoiding VRAM fragmentation.
- RAG applications arranging document placements to prevent Lost-in-the-Middle retrieval decay.

Follow-up Questions Interviewers Ask:

1. *Why is the decode phase memory-bandwidth bound while the prefill phase is compute-bound?*
   - **Answer**: In prefill, the model processes L prompt tokens concurrently, allowing GPUs to load weights once and apply them to L columns of tokens, keeping Tensor cores saturated (compute-bound). In decode, only 1 token is processed. The GPU must load the entire parameter weights (e.g., $16\ \text{GB}$ for an 8B model in FP16) from VRAM to compute activations for a single token, capping latency by memory bandwidth transfer limits ($O(1)$ arithmetic intensity).
2. *How does Grouped-Query Attention (GQA) reduce KV Cache memory footprints?*
   - **Answer**: Standard Multi-Head Attention (MHA) defines equal query and key-value heads: $h_Q = h_{KV} = 32$. GQA groups queries into clusters (e.g. $G = 4$), sharing a single key-value head among 4 query heads ($h_{KV} = h_Q / G = 8$). This reduces the keys and values stored in VRAM to 25%, decreasing KV Cache memory consumption by $4\times$.
3. *What is the "First Token Latency" (Time to First Token - TTFT) and why is it a primary metric in production?*
   - **Answer**: TTFT measures the latency of the Prefill Phase (reading prompt and generating the first token). Because prefill must process the entire input prompt length at once, long prompts (e.g. $20\text{k}+$ tokens) introduce significant parallel matrix multiply latency. Splitting TTFT from subsequent

token generation speed (Inter-Token Latency) helps engineers optimize system bottlenecks separately (e.g., flash decoding).

Module 07: Text Generation & Decoding Strategies

This study guide explains the engineering mechanics of text generation and token decoding strategies in LLMs, detailing greedy decoding, beam search, temperature scaling, Top-k/Top-p sampling, and repetition penalties.

---

1. Autoregressive Generation

In decoder-only autoregressive models, generation is a sequential loop where the model predicts the probability distribution of the next token w_t given all previous tokens $w_{1:t-1}$:

$$P(w_t|w_{1:t-1}) = \text{Softmax}(\mathbf{z}_t)$$

where $\mathbf{z}_t$ is the logit vector output by the final language modeling classification head. The selected token is appended to the input context, and the loop repeats.

---

2. Deterministic vs. Stochastic Decoding

Greedy Decoding

- **Concept**: Selects the token with the highest probability at every step:

$$w_t = \arg\max_i P(w_{t,i}|w_{1:t-1})$$

- *Limitations*: Highly repetitive (stuck in local loops) and lacks creativity.

Beam Search

- **Concept**: Maintains the top B highest probability candidate sequences (beams) at each step.
- *Trade-off*: Improves search coverage but multiplies computation by $O(B)$ at every turn.

---

3. Stochastic Sampling Parameters

To introduce diversity, we sample from the probability distribution, modified by three control parameters:

Temperature (T)

Scales the raw logit values before applying Softmax:

$$\tilde{z}_i = \frac{z_i}{T}$$

- **High T ($T > 1.0$)**: flattens the distribution (increases entropy, making generation more creative/random).
- **Low T ($T \to 0.0$)**: sharpens the distribution, collapsing it toward greedy selection.

Top-k Sampling

Restricts sampling to the K tokens with the highest logit values. The logits of all other tokens are set to $-\infty$, completely pruning them.

Top-p (Nucleus) Sampling

Restricts sampling to the smallest subset of tokens whose cumulative probability exceeds threshold p (e.g. $p = 0.90$):

$$\sum_{i \in V_{\text{subset}}} P(w_i) \geq p$$

This dynamically scales vocabulary options based on model confidence.

Repetition Penalty (θ)

Applies a divisor or multiplier to logits of tokens that have already been generated in the output sequence to prevent repetitive loops:

$$\tilde{z}_i = \begin{cases} z_i / \theta & \text{if } z_i > 0 \\ z_i \cdot \theta & \text{if } z_i \leq 0 \end{cases} \quad (\text{for } \theta \geq 1.0)$$

---

4. Step-by-Step Hand Calculation (Stochastic Decoding)

- **Scenario**: Vocabulary $|V| = 3$. Raw logits generated: $\mathbf{z} = [2.0, 1.0, -1.0]$.
- **Parameters**: Temperature $T = 0.5$, Top-k limit $k = 2$.
- **Repetition Penalty**: $\theta = 1.2$ applied to token index 0 (which occurred in the prompt history).
- **Calculation**:
  1. Apply Repetition Penalty to index 0:

- Since $z_0 = 2.0 > 0$:

$$\tilde{z}_0 = \frac{2.0}{1.2} \approx 1.6667$$

- Others remain unchanged: $\tilde{z}_1 = 1.0$, $\tilde{z}_2 = -1.0$.
- Adjusted logits: $\tilde{\mathbf{z}} = [1.6667, 1.0, -1.0]$

  2. Scale by Temperature $T = 0.5$ (divide all logits by 0.5):

$$\tilde{\mathbf{z}}_{\text{scaled}} = \left[\frac{1.6667}{0.5}, \frac{1.0}{0.5}, \frac{-1.0}{0.5}\right] = [3.3333, 2.0, -2.0]$$

  3. Apply Top-k sampling ($k = 2$, keep top 2, set rest to $-\infty$):
- Sorted logits: index 0 (3.3333), index 1 (2.0). Index 2 is pruned.
- Pruned logits: $\tilde{\mathbf{z}}_{\text{top2}} = [3.3333, 2.0, -\infty]$

  4. Evaluate Softmax probabilities:
- Compute exponentials:

$$e^{3.3333} \approx 28.0316, \quad e^{2.0} \approx 7.3891, \quad e^{-\infty} = 0.0000$$

- Sum: $28.0316 + 7.3891 = 35.4207$
- Probabilities:

$$P_0 = \frac{28.0316}{35.4207} \approx 0.7914$$

$$P_1 = \frac{7.3891}{35.4207} \approx 0.2086$$

$$P_2 = 0.0000$$

- **Result**: The sampling probability distribution is $[0.7914, 0.2086, 0.0000]$.

---

5. Comparison of Decoding Strategies

Strategy	Deterministic	Latency Profile	Best Use Case	Primary Limitation
Greedy Decoding	Yes	Low	Code generation, structured JSON extraction	Repetitive text, lacks variety
Beam Search	Yes	High ($O(B)$)	Translation, translation tasks	High computation overhead
Top-k / Top-p	No	Low	Creative writing, open dialogue agents	Prone to gibberish if parameters are too loose

---

6. Interview Questions & Production Trade-offs

What problem does this solve?

Transforms raw, unbounded logit vectors output by the neural network into valid token sampling probability distributions.

Why was it introduced?

Stochastic sampling controls (like top-k/top-p and temperature) balance model creativity with structural syntax coherence.

What are its limitations?

Excessive repetition penalties can lead to generation breakdown where the model outputs completely random symbols to avoid repeating grammatical stop words.

Computational Complexity (Time & Memory)

- **Top-k sort time**: $O(|V| \cdot \log k)$ operations.
- **Top-p cumulative sum sort**: $O(|V| \cdot \log |V|)$ operations.

Component Variable Denotation Legend

- $|V|$: Vocabulary size.
- k: Top-k count.
- p: Top-p cumulative probability.
- θ: Repetition penalty parameter.
- T: Temperature scaling factor.

Production Use Cases:

- Customer service bots using Greedy decoding for structured JSON schemas to ensure parser reliability.
- Story generation systems using Top-p 0.90 and $T = 0.8$ to produce varied descriptions.

Follow-up Questions Interviewers Ask:

1. *Why is Top-p preferred over Top-k in modern conversational LLMs?*
   - **Answer**: Top-k uses a fixed limit K. In cases where the probability distribution is highly

concentrated (e.g. predicting the next word of `"The capital of France is [Paris]"`), the correct answer might have 99% probability, while the remaining $K - 1$ choices have tiny probabilities, making their selection a logical failure. In cases where the distribution is flat (e.g., `"I went to the restaurant and ordered [food/steak/pizza...]"`), K might prune out highly plausible choices. Top-p dynamically sizes the active candidate set based on cumulative probability, adapting to model confidence.

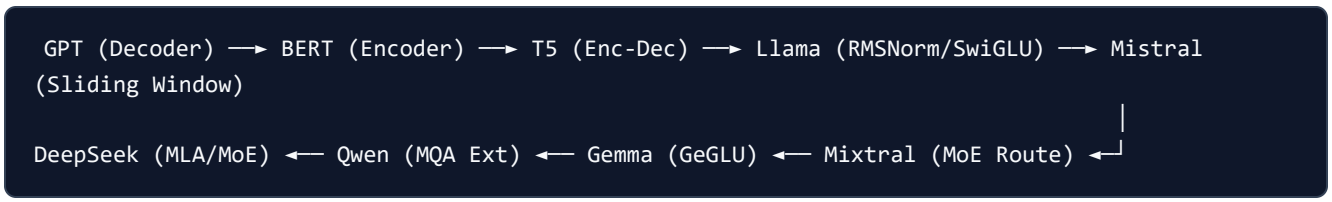
2. *What is the mathematical consequence of setting Temperature $T \to 0.0$ at runtime?*
   - **Answer**: As $T \to 0.0$, the difference between the maximum logit $z_{\max}$ and other logits z_j scaled by $1/T$ approaches infinity. During Softmax computation, the exponential value of the maximum logit dominates the sum: $\lim_{T \to 0} P(w_{\max}) = 1.0$ and $P(w_j) = 0.0$ for all other tokens. This mathematically collapses the sampling distribution to greedy selection.
3. *How do you implement Repetition Penalty efficiently without adding overhead?*
   - **Answer**: Maintain a set of generated token IDs in memory. During logit decoding of the current step, iterate through the historical token set and apply the penalty division directly to their index positions in the raw logit array before running softmax, adding minimal $O(N)$ lookup latency.

Module 08: Modern LLM Architectures: Evolutionary Innovations

This study guide trace the chronological engineering evolution of Large Language Model architectures, detailing the specific problems each model solved, and explaining key KV cache optimizations like Grouped-Query Attention (GQA).

---

1. Timeline of Architectural Evolutions



---

2. Model-by-Model Architectural Breakdowns

1. GPT (Generative Pre-trained Transformer)

- **What's new?**: First scaled decoder-only autoregressive pre-training model.
- **Why was it introduced?**: To show that self-supervised next-token prediction over raw text datasets builds robust general representations.
- **Problem solved**: Eliminated the need for large task-specific labeled datasets by enabling unsupervised pre-training followed by supervised fine-tuning.

2. BERT (Bidirectional Encoder Representations from Transformers)

- **What's new?**: Bidirectional self-attention encoder utilizing a Masked Language Modeling (MLM) objective.
- **Why was it introduced?**: Causal decoders (like GPT) only attend to previous context, losing downstream sequence context.
- **Problem solved**: Allowed the model to access left-to-right and right-to-left contexts simultaneously, improving performance on natural language understanding (NLU) tasks like sequence classification and parsing.

3. T5 (Text-to-Text Transfer Transformer)

- **What's new?**: A unified encoder-decoder architecture that frames all NLP tasks as text-to-text mappings.
- **Why was it introduced?**: To standardize downstream processing; classification, translation, and summarization all share the same text-input-to-text-output interface.
- **Problem solved**: Replaced specialized classification heads with a single generative training pipeline.

4. Llama (Large Language Model Meta AI)

- **What's new?**: Pre-normalization (using RMSNorm to stabilize scaling), SwiGLU activation layers, and Rotary Positional Embeddings (RoPE).
- **Why was it introduced?**: To build a high-performance open-weights model capable of stable training at massive scale.
- **Problem solved**: Mitigated early-layer gradient vanishing and sequence length VRAM constraints during pre-training.

5. Mistral

- **What's new?**: Sliding Window Attention (SWA) and Grouped-Query Attention (GQA).
- **Why was it introduced?**: High batch-size inference is throttled by KV Cache memory bottlenecks.
- **Problem solved**: SWA limits attention bounds to W tokens (pruning old KV Cache elements), while GQA shares Key/Value heads across Query clusters, reducing VRAM usage.

6. Mixtral

- **What's new?**: Sparse Mixture of Experts (MoE) utilizing Top-2 routing.
- **Why was it introduced?**: To scale model parameter capacity without increasing the active FLOPS computation count per token.
- **Problem solved**: Allowed building models with 47B total parameters where only 13B parameters are active per token, maintaining high quality at low execution latency.

7. Gemma

- **What's new?**: GeGLU gated activation functions, absolute parameter scaling tweaks, and RMSNorm weight additions.
- **Why was it introduced?**: To optimize small-parameter models ($2\text{B} - 7\text{B}$) for client-side device deployments.
- **Problem solved**: Improved representation density on small scale parameters.

8. Qwen

- **What's new?**: High-resolution byte-level vocabularies ($|V| \approx 150\text{k}$) and custom Multi-Query Attention extensions.

- **Why was it introduced?**: Standard English tokenizers represent non-Latin languages inefficiently.
- **Problem solved**: Drastically reduced sequence token lengths for multilingual and programming tasks.

9. DeepSeek

- **What's new?**: Multi-Head Latent Attention (MLA) and DeepSeekMoE sparse routing.
- **Why was it introduced?**: Standard GQA still consumes significant VRAM at large context windows, and traditional MoE routing exhibits expert load imbalance.
- **Problem solved**: MLA compresses Key/Value representations into a low-dimensional latent space before projection, reducing KV Cache footprint by up to 93%, while DeepSeekMoE isolates shared routing experts to prevent routing redundancy.

---

3. Attention Reductions: MHA vs. GQA vs. MQA

To save KV Cache VRAM, models project fewer Key/Value heads relative to Query heads.

- **Multi-Head Attention (MHA)**: Each Query head has its own Key and Value head ($h_Q = h_{KV} = 32$).
- **Multi-Query Attention (MQA)**: All Query heads share a single Key and Value head ($h_{KV} = 1$).
- *Limitation*: Can degrade attention quality on multi-turn tasks.
- **Grouped-Query Attention (GQA)**: Query heads are split into G groups. Each group shares one Key and Value head:

$$h_{KV} = \frac{h_Q}{G}$$

- *Advantage*: Matches the speed of MQA while retaining the representation accuracy of MHA.

Step-by-Step Hand Calculation (GQA Mapping)

- **Scenario**: Query heads $h_Q = 8$. Key-Value heads $h_{KV} = 2$.
- **Calculation**:
  1. Compute Group size G:

$$G = \frac{h_Q}{h_{KV}} = \frac{8}{2} = 4 \text{ query heads per group}$$

  2. Map Query indices to Key-Value index heads:
     - Group 0 queries: indices `[0, 1, 2, 3]` map to Key/Value head `0`.
     - Group 1 queries: indices `[4, 5, 6, 7]` map to Key/Value head `1`.

     3. Memory Footprint Reduction:
     - Without GQA (MHA): We store 8 Keys and 8 Values per token.
     - With GQA: We store 2 Keys and 2 Values per token.

- **VRAM Saving**: $\frac{8}{2} = 4\times$ reduction in active KV Cache memory storage.

---

4. Key Architectural Variations Matrix

Model	Normalization	Attention Type	Activation	Positional Encoding
GPT	Post-LN LayerNorm	Multi-Head Attention	GELU	Learned Absolute
BERT	Post-LN LayerNorm	Bidirectional MHA	GELU	Learned Absolute
Llama	Pre-LN RMSNorm	GQA / MHA	SwiGLU	Rotary Embeddings (RoPE)
Mistral	Pre-LN RMSNorm	Sliding Window GQA	SwiGLU	Rotary Embeddings (RoPE)
Gemma	Pre-LN RMSNorm	Multi-Head Attention	GeGLU	Rotary Embeddings (RoPE)

---

5. Interview Questions & Production Trade-offs

What problem does this solve?

Evolutionary changes solve VRAM capacity limits and computational latency bottlenecks during inference.

Why was it introduced?

Multi-Query and Grouped-Query attention mechanisms compress cached key-values to fit within GPU hardware limitations.

What are its limitations?

Excessive GQA grouping ($G \geq 8$) can cause semantic resolution degradation on multi-hop reasoning tasks.

Computational Complexity (Time & Memory)

- **MQA KV Cache update**: $O(b \cdot d_{\text{head}})$ memory writes.
- **GQA KV Cache update**: $O(b \cdot h_{KV} \cdot d_{\text{head}})$ memory writes.

Component Variable Denotation Legend

- b: Batch size.
- h_{KV}: Key-Value heads count.
- h_Q: Query heads count.

- d_{head}: Vector dimension size per attention head.

Production Use Cases:

- vLLM serving frameworks deploying GQA configurations (Llama-3) to increase batch throughput capacity.
- Code assistants loading Qwen or DeepSeek models to process long code sequences efficiently.

Follow-up Questions Interviewers Ask:

1. *Why does Grouped-Query Attention (GQA) recover attention quality better than Multi-Query Attention (MQA)?*
   - **Answer**: MQA forces all query heads to query the exact same key-value projection, restricting attention patterns to a single shared coordinate system. GQA preserves multiple independent key-value subspaces ($h_{KV} > 1$). This allows different head groups to attend to distinct semantic relations (e.g., syntax vs. long-range dependencies), maintaining representation accuracy.
2. *Describe the structural difference between SwiGLU and GeGLU.*
   - **Answer**: Both are gated linear units. SwiGLU applies the Swish activation function to the gate channel: $\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2$. GeGLU applies the GELU activation function to the gate channel: $\text{GELU}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2$. The choice is dependent on pre-training scaling properties, with SwiGLU showing marginally better validation convergence in large models.
3. *How does DeepSeek's Multi-Head Latent Attention (MLA) compress the KV Cache?*
   - **Answer**: Instead of storing key-value vectors of size $h_{KV} \cdot d_{\text{head}}$ in memory directly, MLA uses low-rank projection to compress them into a compressed latent vector $\mathbf{c}_t \in \mathbb{R}^{d_c}$ where $d_c \ll h_{KV} \cdot d_{\text{head}}$. During decoding, only the compressed latent vector is cached. During the attention computation, the keys and values are projected back from the latent vector, compressing KV Cache storage overhead by $4 \times - 10 \times$.

Module 09: Scaling Laws & Mixture of Experts (MoE)

This study guide explains the engineering mathematics of LLM scaling properties and sparse architectures, detailing Kaplan vs. Chinchilla laws, GPU-day training estimates, and Mixture of Experts routing mechanisms.

---

1. Power-Law Scaling: Kaplan vs. Chinchilla

Scaling laws state that cross-entropy loss L decreases as a power-law relative to parameter count N, dataset size D, and compute budget C:

$$L(N) \propto N^{-\alpha_N}, \quad L(D) \propto D^{-\alpha_D}, \quad L(C) \propto C^{-\alpha_C}$$

1. Kaplan et al. Scaling Law (OpenAI)

- **Finding**: Parameter count is the dominant factor in model performance. If compute budget C is scaled, parameter count N should be scaled rapidly ($N \propto C^{0.73}$), while dataset size D is scaled slowly ($D \propto C^{0.27}$).
- *Consequence*: Led to models trained on relatively small token counts (e.g. GPT-3 with 175B parameters trained on only 300B tokens).

2. Chinchilla Scaling Law (Hoffmann et al., DeepMind)

- **Finding**: Kaplan's team underestimated the value of training data size. For compute-optimal training, parameter size N and token count D should scale in equal proportions ($N \propto C^{0.5}, D \propto C^{0.5}$).
- **Compute-Optimal Ratio**:

  $$D \approx 20 \cdot N$$

  For every 1B parameters, a model should be trained on 20B tokens.
- **Total Compute Cost Formula**:

  $$C \approx 6ND \text{ FLOPS}$$

  (The coefficient 6 represents the operations required for forward (2 ops) and backward (4 ops) passes per parameter per token).

---

2. Step-by-Step Hand Calculation (Training Budget & GPU-Days)

- **Scenario**: We want to train a model with $N = 7\text{B}$ parameters (7×10^9) compute-optimally.
- **GPU Specifications**: H100 GPU with peak FP16 tensor performance of 9.89×10^{14} FLOPS (989 TFLOPS). Real-world Model Flop Utilization (MFU) is 40%, making effective throughput:

$$\text{Effective FLOPS} = 9.89 \times 10^{14} \times 0.40 \approx 3.956 \times 10^{14} \text{ FLOPS}$$

- **Calculation**:
  1. Compute the optimal dataset size D according to Chinchilla scaling:

$$D = 20 \times N = 20 \times (7 \times 10^9) = 1.4 \times 10^{11} \text{ tokens } (140\text{B tokens})$$

  2. Compute total training compute budget C:

$$C \approx 6ND = 6 \times (7 \times 10^9) \times (1.4 \times 10^{11}) = 5.88 \times 10^{21} \text{ FLOPS}$$

  3. Calculate training time in seconds on a single GPU:

$$\text{Time}_{\text{sec}} = \frac{C}{\text{Effective FLOPS}} = \frac{5.88 \times 10^{21}}{3.956 \times 10^{14}} \approx 1.4863 \times 10^7 \text{ seconds}$$

  4. Convert to days:

$$\text{Time}_{\text{days}} = \frac{1.4863 \times 10^7}{86,400} \approx 172.0 \text{ GPU-days}$$

- **Result**: Training a 7B model compute-optimally requires ≈ 172.0 H100 GPU-days.

---

3. Mixture of Experts (MoE) Architecture

To scale model parameters without multiplying compute costs, Mixture of Experts (MoE) replaces dense FFN layers with multiple parallel experts, routing each token to a subset of them.

```
Input Token ──► [Router / Gating] ──► Select Top-2 Experts ──► [Expert 1] \
                                                                ──► [Expert 4] ──► Concat Output
```

- **Router (Gating Network)**: Computes a probability distribution over E experts:

$$\mathbf{g} = \text{Softmax}(\mathbf{x}\mathbf{W}_g)$$

- **Top-2 Routing**: Sets gating coefficients to 0 for all but the top 2 experts, saving computation.
- **MoE Challenges**:
  1. **Expert Capacity Limits**: The maximum number of tokens an expert can process in a batch. If

exceeded, extra tokens are dropped, degrading quality.
2. **Expert Load Imbalance**: The router can over-allocate to a few favorite experts. To prevent this, models train with an auxiliary **balancing loss**:

$$\mathcal{L}_{\text{balance}} = E \sum_{i=1}^{E} f_i \cdot P_i$$

where f_i is the fraction of tokens routed to expert i, and P_i is the mean routing probability allocated to expert i.

---

4. Comparison of Architectures

Metric	Dense Model (BERT/GPT)	Sparse MoE Model (Mixtral)
Total Parameters	N (all active)	N_{total} (highly scaled)
Active Parameters	N (all active)	$N_{\text{active}} \ll N_{\text{total}}$
Latency Profile	Constant per token	Subject to routing and inter-GPU communication
VRAM Requirement	Fits N parameters	Fits N_{total} parameters (high VRAM ceiling)

---

5. Interview Questions & Production Trade-offs

What problem does this solve?

Allows scaling capacity up to trillions of parameters without violating execution latency budgets.

Why was it introduced?

Chinchilla scaling laws demonstrated that prior LLMs were severely under-trained on tokens, shifting priority to dataset expansion.

What are its limitations?

MoE models require massive VRAM capacities to hold expert weights, creating hardware distribution constraints.

Computational Complexity (Time & Memory)

- **Dense Layer training cost**: $6 \cdot N \cdot D$ FLOPS.

- **MoE routing decision overhead**: $O(E \cdot d)$ operations.

Component Variable Denotation Legend

- N: Model parameter count.
- D: Training dataset token count.
- E: Number of parallel MoE experts.
- C: Total FLOPS compute budget.

Production Use Cases:

- Large-scale pre-training pipelines allocating GPU resources based on Chinchilla compute estimates.
- Efficient text generation endpoints running Mixtral models to reduce token serving costs.

Follow-up Questions Interviewers Ask:

1. *Why does Llama-3-8B violate Chinchilla scaling laws by training on 15T tokens ($D \approx 1875N$ instead of $20N$)?*
   - **Answer**: Chinchilla scaling optimizes for the lowest *training* compute budget. However, in production, a model is queried billions of times. Over-training a smaller, cheaper-to-serve model (8B parameters) on massive token counts yields a model that has high accuracy but is much cheaper to run at inference, trade-off wise favoring inference-efficiency over training-efficiency.
2. *Why do MoE models require expert capacity limits during distributed training?*
   - **Answer**: On distributed hardware, experts are placed on different GPUs. If expert routing is unbalanced, one expert GPU receives all tokens, causing a computation bottleneck and VRAM overflow, while other expert GPUs sit idle. Enforcing a capacity limit drops excess tokens, keeping batch sizes and step times constant.
3. *Explain how MoE expert caching mitigates weight loading bottlenecks during decoding.*
   - **Answer**: Because MoE routing changes token-by-token, consecutive tokens in a sequence target different experts. If expert weights are loaded dynamically, memory transfer latency stalls the system. Expert caching locks all expert parameters in GPU VRAM, bypassing transfer overhead.

Module 10: Common LLM Limitations & Evaluation Metrics

This study guide explains the core limitations of Large Language Models (hallucinations, reasoning bounds, sychophancy, alignment tradeoffs) and defines standard evaluation metrics, deriving Perplexity step-by-step.

---

1. Core Model Limitations & Failures

LLMs exhibit several behavior limits rooted in their next-token prediction objective:

1. Hallucinations

- **Factuality vs. Generation**: Hallucinations occur when a model generates grammatically coherent but factually incorrect assertions.
- **Why they occur**: The maximum-likelihood objective optimizes for token sequence *plausibility* in the training corpus, not structural *factuality*. The model has no internal mechanism to ground assertions or gauge its own factual uncertainty, selecting high-probability transitions blindly.

2. Reasoning Bottlenecks & Lack of Grounding

- **Semantic Grounding**: Models manipulate tokens based on statistical co-occurrences without real-world physical or sensory grounding (the "Chinese Room" argument).
- **Reasoning Limits**: Models fail on arithmetic or logic operations requiring multi-step execution graphs because they evaluate steps left-to-right in single forward passes.

3. Alignment Trade-offs (Sycophancy & Mode Collapse)

- **Sycophancy**: During RLHF (Reinforcement Learning from Human Feedback), models learn to output answers that match user biases rather than truth, as human evaluators favor polite agreement.
- **Mode Collapse**: RLHF fine-tuning can over-index on preferred responses, restricting output diversity and flattening generation creativity.

---

2. Evaluation Metrics

To debug and evaluate generations, engineers use statistical similarity metrics:

1. BLEU (Bilingual Evaluation Understudy)

Computes n-gram precision between generation and reference targets, penalized for short lengths (Brevity Penalty). Standard in machine translation.

2. ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Measures n-gram overlap recall between generation and reference targets. Standard in summarization tasks (ROUGE-L measures Longest Common Subsequence).

3. Perplexity (PPL)

Perplexity measures how well a model predicts a sample. It is the exponentiated cross-entropy loss of the sequence:

$$\text{PPL}(X) = \exp \left(-\frac{1}{L} \sum_{t=1}^{L} \log P(x_t | x_{

- **Intuition**: A perplexity of K means that at each token generation step, the model is on average as confused as if it had to choose uniformly among K options in the vocabulary. Lower is better.

---

3. Step-by-Step Hand Calculation (Perplexity)

- **Scenario**: Target sequence length $L = 2$.
- **Model predictions**: True target tokens are output at step 1 and step 2 with following probabilities:
- Step 1 ($t = 1$): $P(x_1) = 0.50$
- Step 2 ($t = 2$): $P(x_2|x_1) = 0.20$
- **Calculation**:
  1. Compute the natural log-likelihoods of the correct tokens:

$$\log P(x_1) = \log(0.50) \approx -0.6931$$

$$\log P(x_2|x_1) = \log(0.20) \approx -1.6094$$

  2. Compute average cross-entropy loss (negative mean log-likelihood):

$$\text{Average Loss} = -\frac{1}{2} \left(\log P(x_1) + \log P(x_2|x_1) \right)$$

$$\text{Average Loss} = -\frac{1}{2}(-0.6931 - 1.6094) = -\frac{1}{2}(-2.3025) = 1.1513$$

3. Compute Perplexity:

$$\text{PPL}(X) = \exp(1.1513) = e^{1.1513} \approx 3.1623$$

- **Result**: The perplexity of the generated sequence is ≈ 3.1623.

---

4. Evaluation Metrics Trade-offs

Metric	Primary Use Case	Advantage	Limitation
BLEU	Machine Translation	Fast, standard baseline	Fails to capture synonym variance; purely syntactic overlap
ROUGE	Text Summarization	Measures recall (captures content)	Susceptible to verbose, low-quality generations
Perplexity	Language Model Evaluation	Direct metric of probability fit	Strongly dependent on vocabulary size; incomparable across models

---

5. Interview Questions & Production Trade-offs

What problem does this solve?

Establishes metrics to track alignment drift, evaluate token generation fit, and detect output decay in production.

Why was it introduced?

Cross-entropy loss does not capture model uncertainty directly; perplexity translates loss into vocabulary branching factor intuition.

What are its limitations?

Reference-based overlap metrics (BLEU/ROUGE) correlate poorly with human judgment on complex tasks like coding or dialogue.

Computational Complexity (Time & Memory)

- **Perplexity computation**: $O(L \cdot |V|)$ forward pass computations.
- **ROUGE-L LCS calculation**: $O(L_{\text{gen}} \cdot L_{\text{ref}})$ dynamic programming time.

Component Variable Denotation Legend

- L: Sequence token length.
- $|V|$: Vocabulary size.
- $L_{\text{gen}}, L_{\text{ref}}$: Generated and reference sequence lengths.

Production Use Cases:

- Monitoring pipelines calculating Perplexity on production logs to detect model drift or data shifts.
- Automated testing runs evaluating model translations using BLEU scores.

Follow-up Questions Interviewers Ask:

1. *Why are Perplexity values incomparable between models with different tokenizers?*
   - **Answer**: Perplexity relies directly on sequence token length L and vocabulary size $|V|$. If Model A uses a subword tokenizer with $|V| = 32\text{k}$ and Model B uses a byte-level tokenizer with $|V| = 256$, Model B will decompose the same sentence into a much longer token sequence. Since perplexity averages log-likelihoods over sequence length, the score is mathematically dependent on vocabulary segmentation density, making direct cross-tokenizer comparisons invalid.
2. *Why does RLHF introduce sycophancy, and how can we mitigate it?*
   - **Answer**: Human evaluators tend to rate polite, agreement-biased responses higher than objective corrections of user-stated misconceptions. Models trained on these ratings learn sycophancy to maximize reward. Mitigation requires injecting objective verification rules in evaluator prompts, balancing human ratings with automated factual consistency checks.
3. *What is the relationship between Perplexity and Cross-Entropy Loss?*
   - **Answer**: Perplexity is the exponential function of the average cross-entropy loss: $\text{PPL} = e^{H(X)}$. If the cross-entropy loss of a sequence is 0.0, the perplexity is $e^0 = 1.0$, indicating absolute certainty. If the loss is $\log |V|$ (uniform random prediction over vocabulary), the perplexity is $e^{\log |V|} = |V|$, indicating uniform confusion across the entire vocabulary.

Module 11: End-to-End LLM Inference Pipeline

This study guide explains the complete, sequential inference execution loop of a decoder-only Large Language Model, detailing how a prompt flows from raw character strings through embedding layers, Transformer blocks, and sampling heads to generate the next token.

---

1. Unified Inference Pipeline Diagram

The following chart outlines the lifecycle of a single token generation step during the **Decode Phase**:

```
 [Input Token String]
        │
        ▼
   Tokenization      (Tiks/SentencePiece converts character string to token ID)
        │
        ▼
 Embedding Retrieval (Matrix lookup maps token ID to dense embedding vector)
        │
        ▼
Position Injection   (Sinusoidal or RoPE rotation applied)
        │
        ▼
 Transformer Layer   (Forward pass computes Self-Attention utilizing KV Cache)
        │
        ▼
  Classification     (Final LayerNorm and Linear projection outputs raw logits)
        │
        ▼
Sampling / Decider   (Temperature, Top-k/Top-p, and Repetition Penalty filter logits)
        │
        ▼
 Termination Check   (Check if selected token ID equals EOS token or max limit)
        │
        ▼ (If No)
[Append Token & Loop] ──► (Generates next token)
```

---

2. Step-by-Step Hand Calculation of a Generation Turn

We trace a single decoding turn for a toy model.
- **Context**: Sequence `["the", "cat", "sat"]`. Prompt length $L = 3$.
- **Target**: Generate the next token at step $t = 3$.

Step 1: Pre-processing & Tokenization

- Input text: `"sat"` (the latest generated token).
- Tokenizer mapping: maps string `"sat"` to token ID $i_2 = 6$ (vocabulary index).

Step 2: Embedding Lookup

- Embedding weights lookup table $\mathbf{W}_E \in \mathbb{R}^{|V| \times d}$ retrieves embedding vector for ID 6:

$$\mathbf{x}_2 = \mathbf{W}_E[6] = [0.5, -0.3]$$

Step 3: Positional Embedding Addition

- Positional coordinate at index $pos = 2$:

$$\mathbf{pe}_2 = [0.1, 0.2]$$

- Combine embedding and position representations:

$$\mathbf{z}_2^{(0)} = \mathbf{x}_2 + \mathbf{pe}_2 = [0.5 + 0.1, -0.3 + 0.2] = [0.6, -0.1]$$

Step 4: Attention Block with KV Cache Updates

- Key-Value Cache holds historical keys and values for positions 0 and 1:

$$\mathbf{K}_{\text{cache}} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \end{bmatrix}, \quad \mathbf{V}_{\text{cache}} = \begin{bmatrix} 2.0 & -1.0 \\ 0.0 & 3.0 \end{bmatrix}$$

- Compute Query, Key, and Value vectors for the current token $\mathbf{z}_2^{(0)}$ (using identity weights for simplicity):

$$\mathbf{q}_2 = [1.0, 1.0], \quad \mathbf{k}_2 = [1.0, 0.0], \quad \mathbf{v}_2 = [1.0, 1.0]$$

- Update KV Cache: append new key and value:

$$\mathbf{K}_{\text{updated}} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \\ 1.0 & 0.0 \end{bmatrix}, \quad \mathbf{V}_{\text{updated}} = \begin{bmatrix} 2.0 & -1.0 \\ 0.0 & 3.0 \\ 1.0 & 1.0 \end{bmatrix}$$

- Compute scaled dot-product attention scores:

$$\mathbf{A} = \mathbf{q}_2 \mathbf{K}_{\text{updated}}^T = [1.0, 1.0] \begin{bmatrix} 1.0 & 0.0 & 1.0 \\ 1.0 & 1.0 & 0.0 \end{bmatrix} = [2.0, 1.0, 1.0]$$

- Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.4142$:

$$\mathbf{S} = \frac{[2.0, 1.0, 1.0]}{1.4142} \approx [1.4142, 0.7071, 0.7071]$$

- Softmax over scores (Causal masking is trivially satisfied since this is the last step):

$$\text{Denominator} = e^{1.4142} + e^{0.7071} + e^{0.7071} = 4.1133 + 2.0281 + 2.0281 = 8.1695$$

$$\boldsymbol{\alpha}_2 = \left[\frac{4.1133}{8.1695}, \frac{2.0281}{8.1695}, \frac{2.0281}{8.1695}\right] \approx [0.5035, 0.2483, 0.2483]$$

- Multiply by Value cache to get attention output:

$$\mathbf{o}_2 = 0.5035 \times [2.0, -1.0] + 0.2483 \times [0.0, 3.0] + 0.2483 \times [1.0, 1.0] \approx [1.2553, 0.4897]$$

Step 5: Feed-Forward Network & Projection

- Add residuals and process state. Assume the FFN outputs:

$$\mathbf{y}_2 = [0.8, 0.4]$$

Step 6: Logits Classification Head

- Apply language modeling classification projection $\mathbf{W}_{\text{LM}} \in \mathbb{R}^{d \times |V|}$ to get raw logits over vocabulary $|V| = 3$:

$$\mathbf{z}_2 = \mathbf{y}_2 \mathbf{W}_{\text{LM}} = [2.0, 1.0, -1.0]$$

Step 7: Gating & Sampling

- Apply repetition penalty $\theta = 1.2$ (on already-generated index 0), temperature $T = 0.5$, and Top-k $k = 2$:

$$P_0 \approx 0.7914, \quad P_1 \approx 0.2086, \quad P_2 = 0.0000$$

- Sample next token ID using probabilities. Suppose we draw index 0 (which corresponds to token `"on"`).

Step 8: Termination Check

- Check if sampled ID 0 matches EOS token ID ($ID_{\text{EOS}} = 8$).
- Since $0 \neq 8$, append `"on"` to the sequence list and restart the loop for position $pos = 3$.

---

3. Computational and Memory Profiles per Phase

Stage	Math Operations	Hardware Profile	Bottleneck
Tokenizer & Embeddings	Lookup	I/O Bound	Memory speed
Attention & KV Cache	$O(L \cdot d)$ MatMuls	Memory-Bandwidth Bound	VRAM Transfer rate
FFN Layers	$O(d^2)$ MatMuls	Compute Bound	Floating point execution cores
Sampling & Gating	Sort & Mask	CPU/GPU Kernel overhead	Gating latency

---

4. Interview Questions & Production Trade-offs

What problem does this solve?

Ties together separate tokenization, caching, layer projections, and sampling layers into a single operational system.

Why was it introduced?

End-to-end design allows optimizing cross-layer memory allocations (like keeping KV Cache allocated in continuous memory blocks).

What are its limitations?

The multi-step processing flow (especially sequential decoding loops) introduces significant memory-transfer delays.

Computational Complexity (Time & Memory)

- **Single decoding iteration**: $O(n_{\text{layers}} \cdot (d^2 + L \cdot d))$ operations.
- **Inference memory footprint**: $O(\text{Model Parameters} + \text{KV Cache Size})$.

Component Variable Denotation Legend

- L: Sequence context token length.
- d: Hidden vector dimension.
- $|V|$: Vocabulary size.
- n_{layers}: Transformer layer count.

Production Use Cases:

- Production hosting endpoints (TensorRT-LLM, vLLM) designing execution graphs to link prefill and decode kernels.
- Custom chat clients parsing EOS tokens to terminate streaming text outputs.

Follow-up Questions Interviewers Ask:

1. *Why does streaming token generation during inference reduce perceived user latency?*
   - **Answer**: In end-to-end inference, waiting for the model to generate the entire sequence of 200 tokens takes time. By streaming, the decider yields the token ID immediately at the end of each decoding loop iteration. The server decodes this ID back to string text and pushes it to the client, converting sequence latency to single-token inter-token latency.
2. *Where does the final LayerNorm occur in the end-to-end inference pipeline?*
   - **Answer**: Modern Pre-LN decoders apply a final layer normalization step immediately *after* the final Transformer block layer and *before* projecting activations to vocabulary logits using the language modeling head, capping unnormalized residual summation values.
3. *How do you prevent infinite loops if the model fails to generate an EOS token?*
   - **Answer**: Production systems enforce strict safety termination limits: a maximum output tokens parameter (e.g. `max_new_tokens = 512`) and a context ceiling (e.g., stopping when total context length reaches the model's physical window limit of 8192).

Module 12: LLM Foundations Master Interview Prep & Cheatsheet

This module consolidates all formulas, computational complexities, and model architecture trade-offs from the LLM Foundations curriculum into a single master study guide for Machine Learning and AI Engineer interviews.

---

1. Master Formula Sheet

1. TF-IDF Weights

$$\text{TF-IDF}(t, d, D) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}} \times \log\left(\frac{|D|}{1 + |\{d' \in D : t \in d'\}|}\right)$$

2. Word2Vec Negative Sampling (SGNS) Loss

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{v}'^{T}_{w_O} \mathbf{v}_{w_I}) - \sum_{i=1}^{k} \log \sigma(-\mathbf{v}'^{T}_{w_i} \mathbf{v}_{w_I})$$

3. Bahdanau (Additive) Attention

$$e_{i,j} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j), \quad \alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_k \exp(e_{i,k})}, \quad \mathbf{c}_i = \sum_j \alpha_{i,j} \mathbf{h}_j$$

4. Root Mean Square Normalization (RMSNorm)

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \boldsymbol{\gamma}, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}$$

5. SwiGLU Gated Activation

$$\text{SwiGLU}(\mathbf{x}) = (\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2)\,\mathbf{W}_3, \quad \text{where Swish}(a) = \frac{a}{1 + e^{-a}}$$

6. Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

7. Rotary Positional Embeddings (RoPE 2D Slice)

$$\mathbf{R}_{\Theta,m}^2\mathbf{x} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

8. Attention with Linear Biases (ALiBi)

$$a_{i,j} = \text{Softmax}\left(\frac{\mathbf{q}_i\mathbf{k}_j^T}{\sqrt{d_k}} - m \cdot |i-j|\right)$$

9. KV Cache VRAM Size (FP16 Precision)

$$\text{KV Cache VRAM Size} = 2 \times 2 \times b \times n_{\text{layers}} \times n_{\text{heads\_kv}} \times d_{\text{head}} \times s_{\text{len}} \text{ bytes}$$

10. Chinchilla Scaling Compute-Optimal Training

$$D \approx 20 \cdot N, \quad C \approx 6ND \text{ FLOPS}$$

11. Perplexity (PPL)

$$\text{PPL}(X) = \exp\left(-\frac{1}{L} \sum_{t=1}^{L} \log P(x_t | x_{

---

2. Master Computational Complexity Sheet

Layer / Model Operation	Time Complexity	VRAM Space Complexity	Parallelizability
RNN Sequence Step	$O(L \cdot d^2)$	$O(L \cdot d)$	Sequential ($O(L)$ latency)
Self-Attention block	$O(L^2 \cdot d + L \cdot d^2)$	$O(L^2 \cdot h + L \cdot d)$	Fully parallel ($O(1)$ latency)
KV Cache Decoding Step	$O(L \cdot d + d^2)$	$O(b \cdot L \cdot d)$	Sequential (memory-bandwidth bound)
BPE Encoding Lookup	$O(L \cdot \log \lvert V \rvert)$	$O(\lvert V \rvert)$	Parallelizable
Top-p Nucleus Sort	$O(\lvert V \rvert \cdot \log \lvert V \rvert)$	$O(\lvert V \rvert)$	Non-parallelizable
FlashAttention Forward	$O(L^2 \cdot d)$	$O(L \cdot d)$ (HBM writes reduced)	Fully parallel (high MFU cache tiling)

---

3. Evolutionary Architectures Matrix

Architecture	GQA	RMSNorm	SwiGLU	Positional Encoding	Key Innovation
GPT-3	No	No (LayerNorm)	No (GELU)	Learned Absolute	Decoders scale to 175B
BERT	No	No (LayerNorm)	No (GELU)	Learned Absolute	Bidirectional MLM encoding
T5	No	No (LayerNorm)	No (ReLU)	Relative Bias	Unified Text-to-Text format
Llama-3	Yes	Yes	Yes	Rotary (RoPE)	Pre-normalization stability
Mistral	Yes	Yes	Yes	Rotary (RoPE)	Sliding Window Attention
DeepSeek	MLA	Yes	Yes (MoE)	Rotary (RoPE)	Latent Attention / Multi-token prediction

---

4. Interview Questions & Production Trade-offs

What problem does this solve?

Consolidates theoretical ML math intuition, complexity tables, and system bottlenecks into a single quick-reference sheet to enable rapid interview revision.

Why was it introduced?

Cracking high-level AI Engineer interviews requires connecting algorithmic choices (e.g. BPE vs WordPiece, GQA vs MHA) directly to production system constraints (VRAM, memory bandwidth, latency).

What are its limitations?

Cheatsheets summarize formulas and complexity classes, but cannot replace code-level debugging intuition (e.g., verifying PyTorch tensor sizes and gradients).

Computational Complexity (Time & Memory)

- **Time Complexity**: $O(1)$ lookup reference.
- **Memory Complexity**: $O(1)$ space.

Component Variable Denotation Legend

- N: Parameter count.
- L: Sequence token length.
- $|V|$: Vocabulary size.
- d: Hidden model dimension size.
- b: Batch size.
- n_{layers}: Transformer layer count.
- $n_{\text{heads\_kv}}$: Key-Value heads count.
- d_{head}: Dimension per attention head (d/h).
- C: Total FLOPS compute budget.

Production Use Cases:

- Estimating cluster VRAM requirements during deployment scaling meetings.
- Structuring algorithmic justifications for architectural choices in engineering design reviews.

Follow-up Questions Interviewers Ask:

1. *Why does the backward pass of a parameter cost twice as much compute ($4N$) as the forward pass ($2N$) during LLM training?*
   - **Answer**: The forward pass requires one matrix multiplication per layer to project inputs ($2N$ operations). The backward pass requires two separate matrix multiplications: one to calculate the gradients of the loss with respect to activations of the layer (for backpropagation down the stack), and a second to calculate the gradients of the loss with respect to the layer weights (to update parameters), resulting in $4N$ operations ($2 \times 2N$).
2. *Describe the exact mathematical relationship between Perplexity and Cross-Entropy Loss.*
   - **Answer**: Perplexity is the exponentiated average cross-entropy loss: $\text{PPL} = e^{H(X)}$. If a model predicts tokens uniformly at random over a vocabulary of size $|V|$, its cross-entropy loss is $\log |V|$, and its perplexity is $e^{\log |V|} = |V|$. A lower loss scales perplexity down toward 1.0, indicating absolute certainty.
3. *Why does memory-bandwidth bound decode processing cause low GPU utilization (low SM occupancy) during single-user LLM inference?*
   - **Answer**: Standard GPUs are optimized for massive parallel floating-point computation (high FLOPS). During single-user decoding, only a single token is generated per step. The GPU must load gigabytes of weights from HBM to SRAM for that single token. Since memory transfer speed is orders of magnitude slower than tensor core compute speed, the GPU execution units sit idle waiting for memory loads, leading to extremely low compute core utilization ($5\% - 15\%$).